分类号：TP391　　密　　级：公开

UDC：　　单位代码：10142

沈阳工业大学

# 硕士学位论文

论文题目：中国象棋博弈树搜索算法研究与实现

学　　号：2009449

作　　者：宋兴亮

学科名称：计算机应用技术

2012年02月23日

中国知网 https://www.cnki.net

# 沈阳工业大学硕士学位论文

论文题目：中国象棋博弈树搜索算法研究与实现

作者：宋兴亮

指 导 教 师：田翠华 副教授 单位：沈阳工业大学

协助指导教师： 单位：

单位：

论文提交日期：**2011** 年 **02** 月 **23** 日

学位授予单位：沈 阳 工 业 大 学

中国知网 https://www.cnki.net

A Master's Thesis in Computer Application

# RESEARCH AND IMPLMENTATION ON CHINESE CHESS GAME TREE SEARCH ALGORITHM

**By** Song XingLiang

**Supervisor**：Associate Professor Tian CuiHua

School of Information Science and Engineering

Shenyang University of Technology

Feb 23, 2011

中国知网 https://www.cnki.net

# 独 创 性 说 明

本人郑重声明：所呈交的论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写的研究成果，也不包含为获得沈阳工业大学或其他教育机构的学位或证书所使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中做了明确的说明并表示了谢意。

签名：宋兴高 日期：2012.02.23

# 关于学位论文使用授权的说明

本学位论文作者和指导教师完全了解沈阳工业大学有关保留、使用学位论文的规定，即：学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅；本人授权沈阳工业大学可以将学位论文的全部或部分内容编入有关数据库进行检索、交流，可以采用影印、缩印或其他复制手段保存论文和汇编本学位论文。

（保密的论文在解密后应遵循此规定）

签名：宋兴高 导师签名：田翠华 日期：2012.02.23

中国知网 https://www.cnki.net

# 摘　　要

计算机博弈是人工智能的一个传统研究领域。计算机博弈为人工智能提供一个实验平台，将人工智能的一些理论与方法应用于计算机博弈，可通过博弈水平的高低来检验这些理论与方法的有效性，研究计算机博弈所得到的成果也可推广至人工智能的其他领域。二者相辅相成，相互促进。国际象棋计算机博弈已经比较成熟，历史悠久的中国象棋计算机博弈很多技术还不够成熟，随着对人工智能的深入研究，对中国象棋博弈的研究成为人工智能研究的热点之一。

本文对中国象棋博弈树搜索算法以及博弈系统进行研究，介绍了中国象棋计算机博弈的关键技术，分析了数据结构设计和评估函数在系统中所起的作用。深入研究博弈树的特性以及基于α-β剪枝的博弈树搜索算法的改进策略，包括窗口原则、历史表法、置换表法、空着搜索等，结合不同的改进策略得到不同的搜索引擎，对这些引擎的性能进行测试，验证它们的剪枝效率。研究了静态搜索，在静态搜索中加入吃子走法排序和将军延伸，解决了博弈树搜索的水平效应。

实现一个人机博弈系统，实现生成并显示棋谱，统计搜索引擎每次搜索的叶节点和所用的时间，结合不同的优化策略得到不同的搜索引擎，通过实验证明综合多种改进策略的搜索算法的剪枝效率得到了提高。选取剪枝效率较高的搜索引擎结合静态搜索提高棋力，让这些新的搜索引擎对弈，证明了剪枝效率最高的PVSHNTIQ　搜索算法的棋力也最高。

**关键词：计算机博弈，搜索算法，博弈系统，评估函数**



中国知网　https://www.cnki.net

# Research and Implementation on Chinese Chess Game Tree Search Algorithm


# Abstract

Computer game is a traditional researching field of artificial intelligence(Al).It provide a test platform for Al. Apply some theories and methods of Al in computer game, we can prove the validity of those by means of testing the level of game system. In turn, the fruits obtained through researching the computer game are used in the other realms of Al. So, they supplement and promote each other. Chess computer game has a long history, has acquired great success. Whereas, many techniques of chinese chess computer game are not mature enough, with deeper study on artificial intelligence, research on Chinese chess computer game becomes a hot spot of AI.

In this thesis, chinese chess game tree search algorithm and game systems is researched, introduces the key technology about chinese chess computer game, analysis the effective of evaluation function and data structure design. In-depth study the characteristics of the game tree, and α-β pruning based on the game tree search algorithm improvement strategies, including the principle of the window, transposition table, history heuristic, iterative deepening, null move, etc., combination of different improvement strategies get different search engines, test these engines performance verify their pruning efficiency. Studied the quiescence search, quiescence search and selective extended search avoid the greatly fluctuant evaluation risk that result from changing chessman and check

Implement a human-computer game system, statistical the leaf nodes and time used by search engine, combination of different optimization strategies to get different search engines, through a variety of experiments proof that integrated strategy improved the search algorithm pruning efficiency. Select the search engine which pruning more efficient inject quiescence search to improve playing strength, play game with each other, proved the most efficient pruning search algorithm — PVSHNTIQ playing strength is highest.



**Key Words: computer game, search algorithm, game system, evaluate function**





中国知网 https://www.cnki.net

# 目　　录


摘　　要 ..... I
**Abstract** ..... II
第一章 绪论 ..... 1
1.1 课题的研究背景 ..... 1
1.2 国内外研究现状 ..... 1
1.2.1 国外研究现状 ..... 1
1.2.2 国内研究现状 ..... 2
1.3 论文结构 ..... 3
第二章 数据结构设计 ..... 5
2.1 棋局状态表示 ..... 5
2.1.1 棋盘表示 ..... 5
2.1.2 棋子表示 ..... 7
2.2 走法生成 ..... 8
2.2.1 走法表示 ..... 9
2.2.2 走法生成 ..... 9
2.3 棋局辅助信息 ..... 10
第三章 局面评估函数 ..... 12
3.1 局面评估函数组成部分 ..... 12
3.1.1 固定子力价值 ..... 12
3.1.2 棋子的位置价值 ..... 13
3.1.3 其它评估因素 ..... 13
3.2 局面评估函数与博弈系统的关系 ..... 14
第四章 博弈树搜索算法研究 ..... 16
4.1 博弈树搜索 ..... 16
4.1.1 搜索 ..... 16
4.1.2 中国象棋博弈树 ..... 17
4.2 基本的搜索算法 ..... 18
4.2.1 极大极小值算法 ..... 18
4.2.2 α-β剪枝算法 ..... 21




中国知网 https://www.cnki.net

4.2.3 α-β剪枝算法的性能分析 ..... 25
4.3 基于窗口原则的改进策略 ..... 27
4.3.1 Fail-Soft alpha-beta算法 ..... 27
4.3.2 渴望搜索 ..... 28
4.3.3 极小窗口搜索 ..... 30
4.3.4 基于窗口原则搜索算法小结 ..... 31
4.4 博弈树节点排序 ..... 32
4.4.1 吃子走法启发 ..... 32
4.4.2 杀手启发 ..... 33
4.4.3 历史表启发 ..... 33
4.5 置换表 ..... 34
4.6 迭代加深 ..... 36
4.7 水平效应与静态搜索 ..... 37
4.8 裁剪算法 ..... 38
4.8.1 无害裁剪 ..... 38
4.8.2 空着裁剪 ..... 38
第五章 系统实现与实验分析 ..... 40
5.1 走法生成与局面评估函数 ..... 40
5.2 优化策略实现 ..... 42
5.2.1 历史表实现 ..... 42
5.2.2 置换表实现 ..... 42
5.2.3 迭代加深实现 ..... 44
5.2.4 静态搜索实现 ..... 47
5.3 系统实现 ..... 49
5.3.1 系统功能介绍 ..... 49
5.3.2 综合搜索算法 ..... 51
5.4 实验分析 ..... 55
第六章 结论 ..... 58
参考文献 ..... 59
在学研究成果 ..... 61
致　　谢 ..... 62



中国知网 https://www.cnki.net

# 第一章 绪论

## 1.1 课题的研究背景

1946 年ENIAC诞生，这是第一台数字电子计算机。重 30 吨，主要用于计算弹道和氢弹的研制，虽然它很笨重，但是从那时起人类进入了计算机时代[1]。与计算机相关的众多学科得到了快速发展，试图让计算机具有和人类一样的“思考”能力，也就是具有人工智能，成为一个重要的研究领域。人工智能是一门极富挑战性的科学，从事这项工作的人必须懂得计算机知识，心理学和哲学[2]。人工智能是包括十分广泛的科学，它由不同的领域组成，如机器学习，计算机视觉等等，总的说来，人工智能的目的就是让计算机能够像人一样思考。现在，人工智能已经在进入我们的生活，模糊控制，决策支持等很多方面都有人工智能的影子。人工智能对人类现代生活的影响越来越大，成为人们生活不可分割的一部分。

博弈是指一些个人、团体、或其他组织，在一定的环境条件、一定的约束条件下，依靠所掌握的信息，同时或先后，一次或多次，从各种可能的行为或策略集合中进行选择并实施，各自从中取得相应结果或收益的过程[3]。可见博弈是一种竞争，竞争现象广泛存在于社会活动的许多方面。博弈论在经济、社会、政治、军事等众多领域都已经得到广泛关注和应用。通过计算机和人对弈，如果计算机能够和人一样，在棋类博弈游戏中能够达到一定的竞技水平，就可以说，计算机具备了一定的人工智能。对机器博弈的研究取得的成果不仅仅只用在棋类游戏上[4]，而且也已广泛应用于军事、政治、经济等多个领域，给人类带来了极大的社会效益。中国象棋有着悠久的历史，由于用具简单，趣味性强，成为流行极为广泛的棋艺活动。

## 1.2 国内外研究现状

### 1.2.1 国外研究现状

1997 年卡斯帕罗夫与“深蓝”的“人机对战”始于 5 月 3 日，双方共进行 6 局对弈。在首局比赛中，卡斯帕罗夫执白先行，经过 3 个多小时击败“深蓝”，获得胜利。



中国知网 https://www.cnki.net

在次日举行的第二局比赛中，“深蓝”却以凌厉的攻势战胜卡斯帕罗夫，扳回一局。在接下去的第三、第四和第五局比赛中，双方下得异常激烈，最终均战成平局。11日举行的第六局比赛，“深蓝”充分利用执白先行的好处，一路强攻，仅用一个多小时，双方仅走19步，就让卡斯帕罗夫俯首称臣，取得了决定性的胜利[5]。

2003年卡斯帕罗夫对阵Deep Junior。他和Deep Junior大战六局，卡斯帕罗夫拿下第一局，Deep Junior在第三局追回一分。第四局和，到了第五局和局。然后到了第六局，过了25回合，对局变得火热，然后，Deep Junior 完成了28.f4后，电脑一方提和。令许多人惊奇的是卡斯帕罗夫接受了[6]。计算机每秒运算 10 亿次（搜索到十四层的深度）才能与人棋手争冠世界冠军，深蓝接近了。程序的质量也扮演重要角色。今天的顶级个人电脑程序象Fritz和Junior可以达到并超过每秒处理50万个局面。可以对抗除世界前100名棋手之外的任何人类棋手。在快棋战里人类只有大约前十几位可以胜任，而在超快棋里大概只有两、三名人类棋手能过关[7]。

2005年7月国际象棋大师、英国象棋冠军迈克尔·亚当斯在与超级计算机Hydra的大战中输给了对方。Hydra在这次比赛期间仅仅利用了其64台Pc中的32台。Hydra的每台PC都配置有一个3.06GHz的至强芯片。Hydra每秒钟能够计算2亿步棋，能够预测40步棋[8]。

2010年6月10日，俄罗斯总统的首席经济顾问达瓦科维奇6月9日晚战平国际象棋机器人，比赛结果为1胜1负[9]。达瓦科维奇是俄罗斯国际象棋协会主席，这个机器人手臂是俄罗斯空军工程师克斯坦廷2006年制造的，克斯坦廷是世界顶尖国际象棋女选手亚历山德拉的父亲。

### 1.2.2 国内研究现状

最近十几年，一些高校、单位、个人对中国象棋机器博弈研究投入了大量的工作，出现一大批优秀的象棋软件，如中山大学研究生涂志坚的“纵马奔流”，赵明阳的“象棋奇兵”，黄晨的“象眼”等。其中值得一提的是东北大学的“棋天大圣”，这款软件是由深蓝的设计者之一，号称深蓝之父的许峰雄参与的团队研究开发的，曾在2006年的计算机博弈奥林匹克竞赛中获得冠军[10]。上海机器博弈研究所的黄晨凭个人兴趣开发出来的“象眼”引擎挂在“象棋巫师”的界面上，具有很高的对战能力，可以说是棋力最强的非商业化软件。近二十年来，中国象棋博弈的研究常常通过比赛对弈的



中国知网 https://www.cnki.net

方式来验证各自研究的成果，2004 年起在台湾成功大学举办的世界电脑象棋争霸赛。“浪潮杯”2006 中国计算机博弈锦标赛暨中国象棋人机大战在浪潮集团的鼎力赞助下，在北京中国科技馆、北京奥体中心、北京香格里拉饭店三地拉开战幕，成为中国机器博弈和IT史上的盛话。2007 年第二届赛事成功在重庆举办。2008 年第三届中国赛由国际机器博弈协会主办，成为国际机器博弈史上规模最大、水平最高的一次比赛，期间举办了规模盛大的人工智能、机器博弈高峰论坛，推动了中国机器博弈活动和中国智能计算技术的发展。2009 年赛事于 11 月 18 日至 21 日在由北大、清华、哈工大、中科院深圳先进院组成的深圳大学城举办，促进中国人工智能和高性能计算技术、IT 通讯等智能制造业的发展[11]。

第四届中国计算机博弈锦标赛于 2010 年 11 月 8 日在北京理工大学落下帷幕。设置了中国象棋、围棋、六子棋、九路围棋、亚马逊、苏拉卡尔塔、点格棋、幻影围棋等 8 种棋类项目。本次全国计算机博弈锦标赛由中国人工智能学会机器博弈专业委员会主办，吸引了 110 名选手操纵的 54 个程序参赛，选手来自东北大学等 13 所院校和 5 家知名IT企业[11]。

第五届全国计算机博弈锦标赛于 2011 年 9 月 12 日至 14 日在北京科技大学举办，这次大赛由教育部高等学校计算机科学与技术专业教学指导委员会、中国人工智能学会机器博弈专业委员会联合主办，作为中国计算机博弈领域最高水平的竞赛，大赛吸引了来自全国各所著名的科研所、高等院校的 103 支队伍参加 8 个项目的角逐[12]。

## 1.3 论文结构

第一章介绍了研究背景、国内外研究现状以及论文的章节安排。

第二章介绍了中国象棋博弈系统中的数据结构的设计，介绍了几种常用的棋盘、棋子的表示方法和目前常用的走法生成的方法，重点介绍了课题的博弈系统中棋盘棋子编码和走法表示及生成。

第三章阐述了局面评估函数中涉及的因素，局面评估函数与博弈系统其它因素之间的关系。

第四章介绍了博弈树的特性和人工智能中搜索的相关知识，介绍了中国象棋博弈树的特征，基于极大极小理论的搜索算法。重点研究基于 α-β 剪枝算法的各种优化策略。



中国知网 https://www.cnki.net

第五章介绍了历史表法，置换表法以及迭代深化等搜索算法优化策略在本课题中的实现，完成了人机对弈，计算机之间对弈。通过实验证明应用不同算法优化策略对搜索算法的影响，最终实验表明应用多种优化策略的综合算法的剪枝效率最高棋力最强。

第六章对论文完成的工作进行了分析总结，并指出了需要完善的方面和今后工作重点。



中国知网 https://www.cnki.net

# 第二章 数据结构设计

## 2.1 棋局状态表示

博弈论的基本因素包括：参与人和参与人行为的信息。参与人是指博弈中以自身利益最大化选择行动的决策主体，行为是指参与人的决策变量，如参与人效用最大化决策中的各种因素的估值[13]。对中国象棋人机博弈而言，棋局状态表示就是将和双方博弈情况相关的信息存储在计算机中。对于中国象棋来说，双方的每个兵种的棋子个数，以及棋子所在的位置都需要按照满足象棋规则的固定的格式存在计算机机中，以便在分析阶段分析局面[14]。局面表示和具体博弈事件密切相关，随着博弈事件复杂程度的上升，局面表示的复杂度也会相应的上升。对于棋局状态的表示一个最起码的要求就是能够准确地告诉计算机，在局面中的每一个棋子到底是随于博弈的哪一方属于哪个兵种的棋子，如果连这个都做不到，那么博弈的正确性也就无从谈起。设计数据结构来表示棋局状态需要考虑三个方面：占用空间的数量、操作速度、使用方便与否[15]。棋局状态表示对走法移动、走法生成、博弈树的展开与恢复、局面评估都有着深远的影响。

### 2.1.1 棋盘表示

中国象棋棋盘的初始状态如图 2.1 所示，中国象棋的棋盘是由十条横线和九条纵线构成，走棋的一方面向棋盘其右手边的第一列作为棋谱上的第一列，最左边的那列为第九列，共有 90 个点可以用来放置棋子，共有七个兵种，三十二个棋子，它们在棋盘上分布如图 2.1 所示，棋盘的表示方式有很多种。

对于一般人而言，首先想到是可以用一个 10×9 的二维数组来表示棋盘上的各个落点，这样数组的下标就可以表示行列号。假设我们要表示红方的车所在的位置可以用board[0][0]来表示，依此类推就可以表示棋盘上的每个落点。对于高水平的博弈系统基本不采用这种方式，因为这样做使得移动棋盘上棋子的落点时判断非常繁琐，需要很多额外的代码，而且容易出现错误[16]。



中国知网 https://www.cnki.net

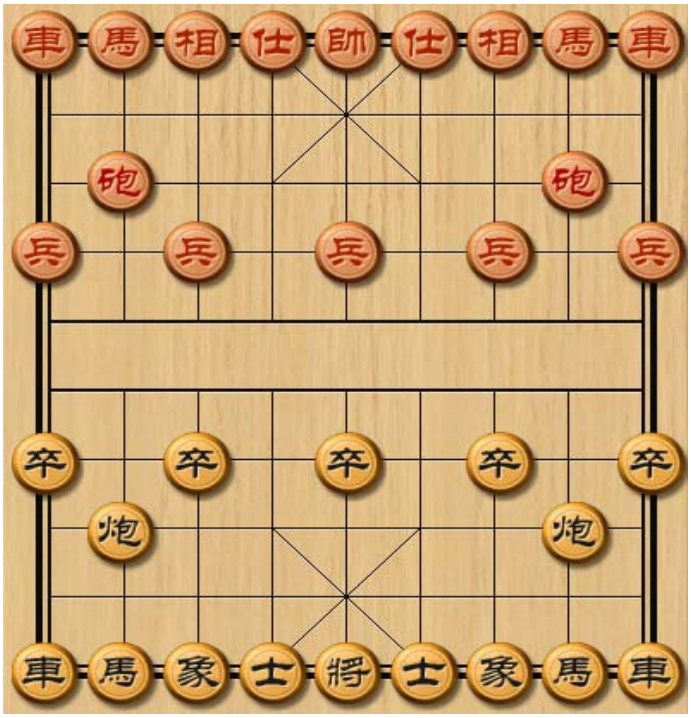


图 2.1 中国象棋的初始棋盘状态
Fig.2.1 Initial state of chinese chessboard

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 |
| 09 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 |
| 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 |
| ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |
| ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |
| 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 | 71 |
| 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 80 |
| 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 |

图 2.2 一维数组的棋盘表示
Fig.2.2 Chessboard denoted by one-dimensional arrays

第二种表示方式是用一个拥有 90 个元素的一维数组来表示棋盘，如图 2.2 所示，棋盘上的落点的编号从 0 到 89，在这种表示棋盘的情况下，要得到行号可以对一维数组的下标整除即N/10，要得到列号可以对一维数组的下标取余即N%10，通过这样的操作就可以到棋子在棋盘上的位置。还有一种表示棋盘的方式就是使用一个大小为



中国知网 https://www.cnki.net

256 的一维数组来表示棋盘上的各个落点[17]。

从占用空间大小、操作方便与否和操作速度几个角度分析以上几种棋盘表示方式。用 10×9 的二维数组来表示棋盘直观形象，易于理解，但是二维数组在查找定位时还需要做一次地址偏移操作，比较费时，而且在局面评估和走法生成时操作将比较繁琐，鉴于以上原因很少有程序采用这种方式。用 0～89 的一维数组来表示棋局状态比用二维数组操作要方便的多，可以用取余和整除来获得位置信息，二者占用的空间也相差不多，在操作速度上二维数组要做一次地址偏移操作速度上慢于一维数组表示方式。

用 0～255 的一位数组来表示棋盘，在占用空间上会大于 0～89 的一维数组，但是就目前的硬件水平来说最一般的配置在一般在 2G 左右，如果用 byte 型的一维数组来表示棋盘的话，这二者之间占用的空间的差距不会对博弈系统产生的影响，可以忽略，如果采用深度优先的博弈树搜索算法只需要保存当前的棋盘状态即可。使用 0～255 的一维数组可以在操作上大量使用移位操作和位操作，不仅代码量少而且每行代码的操作速度更快；方便判断操作是否超出棋盘范围；在判断走棋方的棋子是否过界时也十分方便，只需判断落点的下标的最高位是否有值就可以了。综合以上因素，本文选择 0～255 的一维数组来表示棋盘。

### 2.1.2 棋子表示

中国象棋分为红、黑两方，每方有七种类型的棋子，棋子数为 16 个。在棋盘的每个落点上的状态可以是没有任何棋子也可以是红黑双方的任何一个棋子，这样看来在棋盘上的某个落点的状态有 15 种。

表 2.1 每种棋子编码
Table2.1 Coding type of chessman

| 黑方 | 将 | 士 | 象 | 马 | 车 | 砲 | 卒 |
|---|---|---|---|---|---|---|---|
| 编码 | 16 | 17 | 18 | 19 | 20 | 21 | 22 |
| 红方 | 帅 | 仕 | 相 | 马 | 车 | 炮 | 兵 |
| 编码 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |

棋子的编码主要任务是区分是哪方的哪个棋子同时也要考虑操作和判断的便捷。对棋子的编码有两种不同的方式，一种是只对某一方的某类棋子编码，如表 2.1 所示。



中国知网 https://www.cnki.net

另一种是对棋盘上的每个棋子编码，如表 2.2 所示。

表 2.2 每个棋子编码
Table2.2 Coding type of chessman

| 黑方 | 将 | 士 | 士 | 象 | 象 | 马 | 马 | 车 | 车 | 砲 | 砲 | 卒 | 卒 | 卒 | 卒 | 卒 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 编码 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 红方 | 帅 | 仕 | 仕 | 相 | 相 | 马 | 马 | 车 | 车 | 炮 | 炮 | 兵 | 兵 | 兵 | 兵 | 兵 |
| 编码 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 | 32 |

两种编码方式都可以选择，只要能设计出于编码方式相适应的走法生成数据结构等保证后续操作的正确、方便即可。在课题中使用的是对每种棋子编码的方式，如表 2.1 所示。有时我们只需要判断我们要操作的是哪个种类的棋子，这时就需要对兵种编码而不考虑它是哪一方的棋子。在课题中选择的兵种编码如表 2.3 所示。

表 2.3 棋子种类编码
Table2.3 Chessmam kind coding

| 兵种 | 帅 | 仕 | 相 | 马 | 车 | 炮 | 兵 |
|---|---|---|---|---|---|---|---|
| 编码 | 0 | 1 | 2 | 3 | 4 | 5 | 6 |

结合表 2.1 所示的棋子编码方式，棋子编码与兵种编码可以借助走棋方来进行转换，如果用 0 来表示是红方，chessman&8 就可以得到一个红方棋子的兵种编码，chessman&16 就可以得到一个黑方棋子的兵种编码。由以上的棋盘和棋子编码方案得到的棋盘初始状态如图 2.3 所示。

## 2.2 走法生成

走法生成就是指列出在一个特定的棋局状态下所有的走法，也就是告诉计算机当前局面下一步可以往哪里走的模块，只有执行走法才能改变局面的状态，为搜索算法和局面评估提供服务，走法生成是人机对弈中除棋局状态表示之外另一个基础且不可缺少的部分。可以说在整个对弈的过程中要上百万次的调用这个模块[18]。各种棋类随着规则的不同的不同，走法生成复杂度也不同，比较简单的如五子棋只要棋盘上没有棋子的落点都被视为是可行的走法，这样只要扫描棋盘找到没有棋子的落点即可。中国象棋规则复杂、兵种较多，所以走法生成也很复杂，每个局面下平均有 40 个走法。



中国知网 https://www.cnki.net

```
byte    board[256] = {
 21,20,19,18,17,18,19,20,21, 0,
  0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0,
  0 , 22 , 0, 0 , 0, 0 , 0 , 21, 0 , 0,
 23, 0 ,23, 0 ,23, 0 , 23, 0 , 23, 0,
  0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0,
  0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0,
 15,0 ,15, 0 ,15, 0 ,15, 0 ,15, 0 ,
  0, 14, 0 , 0 , 0 , 0 , 0 , 14 ,0, 0,
  0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0, 0,
 13,12,11,10 ,9 , 10 ,11,12,13 ,0 , };
```

图 2.3 初始局面

Fig.2.3 Initial state of chessboard

### 2.2.1 走法表示

走法生成的代码会被频繁调用，表示走法使用的数据量越小，越简单越好，程序中将棋盘看成是 16×16 的矩阵，那么横纵坐标都只需要 4 个二进制位就可以表示出来，一个 16 位的整型就表示走法的起点和落点，对走法的描述如图 2.4 所示。

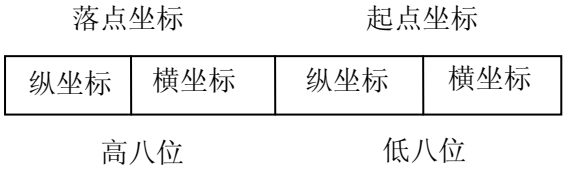



图 2.4 走法表示图

Fig.2.4 Move struct

### 2.2.2 走法生成

在计算机博弈的过程中走法生成通常伴随着搜索进行，并且博弈树搜索过程中调用频繁。像中国象棋这样有复杂规则的棋类，走法的生成将产生大量繁琐的判断[19]。走法生成是程序的性能瓶颈之一。为了提高走法生成的速度，研究人员提出许多方法来对走法生成进行优化。走法生成和具体的棋类规则密切相关，这里说明具有普遍意义的常用方法。

（1）模板匹配法

当计算机找到一个本方的棋子，那么这个棋子的所有可行的落点就已确定，根据这种现实可以为每个兵种设置一个模板，一旦有了起点就可以查找模板得到落点，从



中国知网 https://www.cnki.net

而得到走法[20]，马的走法匹配模板如图 2.4 所示。以本课题中使用的 0～255 的一维数组为例，数组kinghtdes[4][2]={{-33,-31},{-18,14},{-14,18},{31,33}}这八个数组中的数值分别代表相对于起点上、左、右、下四个方向马走法落点的偏移量，与起点做加法就会得到图 2.4 中的圆圈所代表的落点，数组kingdes[4]={-16,-1,1,16}表示相对于起点上、左、右、下四个方向马走法的马腿的偏移量。有了这两个辅助的数据结构再配合再判断一下落点是否在棋盘范围内、是否有己方棋子就能得到马的走法了。模板匹配法适合马和相这样的兵种。

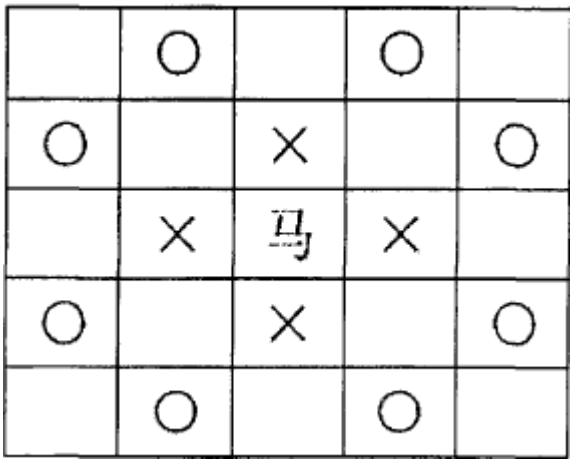



图 2.5 马的走法匹配模板

Fig.2.5 Template matching of knight moving

（2）预置表法

中国象棋中有些兵种的走法生成涉及棋子所在的行和列，典型的是炮和车，这些棋子生产走法时用模板匹配法预置表法不合适，如果用棋盘扫描法又很耗时间，伴随硬件的发展计算机内存已经得到了大幅度提高，如果牺牲空间可以使走法生成的速度提高是可行的，预置表法的思想就是空间换时间，预置表法基于位行、位列技术，具体说就是，为了节省博弈过程中着法生成的扫描时间，把某种棋子在棋盘每个合理位置上，所有可能的棋局状态下的合理着法预先存储起来，生成一个小型数据表集合，称为预置表[21]。当下一次需要时，只需通过查表，便会很快得到可行的着法。它可以将着法生成的速度提高几个数量级。本课题中的炮和车的走法采用预置表法来生成。

## 2.3 棋局辅助信息

除了表示棋盘表示、棋子编码之外，还有一些辅助走法生成的数据结构可以有效



中国知网 https://www.cnki.net

的加速走法的生产速度，如表 2.3 所示，其中红色和暗红色的部分表示的是在棋盘范围内的 90 个落点，其中暗红色表示红黑双方的九宫范围，在本课题中定义一个大小为 byte inboard[256]的一维数组，数组中下标在表 2.3 中被记为红色和暗红色的元素被赋值为 1 表示，如果判断某个位置是否在棋盘范围内就是判断 inboard 数组在这个下标的值是否为 1 即可。定义一个大小为 byte infort[256]的一维数组，数组中下标在表 2.3 中被记为红色和暗红色的元素被赋值为 1 表示，如果如果判断某个位置是否在九宫的范围内就是判断 infort 数组在这个下标的值是否为 1 即可。

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 00 | 01 | 02 | 02 | 04 | 05 | 06 | 07 | 08 | 09 | 0a | 0b | 0c | 0d | 0e | 0f |
| 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 1a | 1b | 1c | 1d | 1e | 1f |
| 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 2a | 2b | 2c | 2d | 2e | 2f |
| 30 | 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 3a | 3b | 3c | 3d | 3e | 3f |
| 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 4a | 4b | 4c | 4d | 4e | 4f |
| 55 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 5a | 5b | 5c | 5d | 5e | 5f |
| 60 | 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 6a | 6b | 6c | 6d | 6e | 6f |
| 70 | 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 7a | 7b | 7c | 7d | 7e | 7f |
| 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 8a | 8b | 8c | 8d | 8e | 8f |
| 90 | 91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 | 9a | 9b | 9c | 9d | 9e | 9f |
| a0 | a1 | a2 | a3 | a4 | a5 | a6 | a7 | a8 | a9 | aa | ab | ac | ad | ae | af |
| b0 | b1 | b2 | b3 | b4 | b5 | b6 | b7 | b8 | b9 | ba | bb | bc | bd | be | bf |
| c0 | c1 | c2 | c3 | c4 | c5 | c6 | c7 | c8 | c9 | ca | cb | cc | cd | ce | cf |
| d0 | d1 | d2 | d3 | d4 | d5 | d6 | d7 | d8 | d9 | da | db | dc | dd | de | df |
| e0 | e1 | e2 | e3 | e4 | e5 | e6 | e7 | e8 | e9 | ea | eb | ec | ed | ee | ef |
| f0 | f1 | f2 | f3 | f4 | f5 | f6 | f7 | f8 | f9 | fa | fb | fc | fd | fe | ff |

图 2.6 扩展的棋盘

Fig.2.6 Expanped chessboard

在给定的局面下生成所有的走法，最多可以有 16 个棋子走，如果要扫描整个 board 数组的 256 个元素显然是没必要的，本课题中设计一个包含 32 个元素的一维数组保存所有的棋子在棋盘上的位置信息，0～15 保存红方棋子的位置信息，16～31 保存黑方棋子的位置信息，当要生成走法时只需要根据走棋方扫描这个数组的相应区域就可以得到所有棋子的位置信息，从而有效提高走法生成的效率。



中国知网 https://www.cnki.net

# 第三章 局面评估函数

计算机在博弈树的搜索过程到达叶节点时，如何判断这个局面是否对自己有利，如果对自己有利又是一个什么程度呢？这就需要对量化分析当前局面。从中国象棋的棋力上分析，博弈树搜索之外，局面评估是最重要的部分，因为对实际局面的评价的好坏，将直接影响今后局势发展的走势。局面评估的过程对具体的棋类知识依赖性也很强，但是仍然有一些一般性的规律可以遵循。目前在计算机象棋博弈中常用的局面评估方法主要采用静态估值方法[22]。将人类棋手好的经验转化为评估函数模型，并对该模型中的参数进行优化，使计算机对棋局的判断更加客观、准确是一个研究课题，用机器学习来解决这一问题是很多研究人员努力的方向。本章分为两节，第一节描述、分析影响局面评估函数的一般性规律，第二节分析局面评估与博弈系统的其他组成部分的关系。

## 3.1 局面评估函数组成部分

### 3.1.1 固定子力价值

固定子力价值，简单说就是根据每个棋子在中国象棋中的重要性和走法的多样性给其赋予一个固定的价值[23]，相对于其它的兵种，车的控制范围最广因而在象棋中它的攻击力最高，应该赋予最大的基本价值，炮和马的固定子力价值大致相当，是车的二分之一左右。相和士的作用相当，用来保护帅的，不具备过河攻击对方的帅能力。根据经验给它们赋予的固定价值为马和炮的二分之一左右。红黑双方的固定子力价值的总和计算方式如公式（3.1）所示。

$$SEVC = \sum CHESSMANi \times NUMofCHESSMANi \tag{3.1}$$

其中参数 CHESSMANi 表示棋盘上的某个棋子的固定基本价值，NUMofCHESSMANi 表示某个兵种在棋盘上棋子的个数。例如兵的子力固定基本价值为 60 棋盘上兵的个数是 3 个那么兵的子力固定基本价值总和是 60，依照兵的计算方式把走棋方的所有棋子的子力固定基本价值做和就得到了本方的子力固定价值。



中国知网 https://www.cnki.net

### 3.1.2 棋子的位置价值

中国象棋中同一个棋子在棋盘上的不同位置对棋局的影响有巨大的差别，一个兵在接近对方九宫时，如果是残局的情况下它的攻击力不逊于一个车。其他的兵种存在相同的问题。这些影响计算机做决策的因素都应该被列为局面评估函数考虑的。由于中国象棋的对称性，一般只给出只给出黑方或者红方的所有兵种的在棋盘上每个位置的价值，要求得另一方的某个棋子的位置价值只要做一个位置镜像就能得到相应棋子的位置价值。以车为例，对车来说它的控制范围的大小和控制区域的不同会影响它的位置价值，所以车在边路的位置价值一般比较小，特别是在边路且不能威胁对方的九宫的情况下。

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 33 | 34 | 35 | 36 | 37 | 38 | 39 | 3a | 3b |
| 43 | 44 | 45 | 46 | 47 | 48 | 49 | 4a | 4b |
| 53 | 54 | 55 | 56 | 57 | 58 | 59 | 5a | 5b |
| 63 | 64 | 65 | 66 | 67 | 68 | 69 | 6a | 6b |
| 73 | 74 | 75 | 76 | 77 | 78 | 79 | 7a | 7b |
| 83 | 84 | 85 | 86 | 87 | 88 | 89 | 8a | 8b |
| 93 | 94 | 95 | 96 | 97 | 98 | 99 | 9a | 9b |
| a3 | a4 | a5 | a6 | a7 | a8 | a9 | aa | ab |
| b3 | b4 | b5 | b6 | b7 | b8 | b9 | ba | bb |
| c3 | c4 | c5 | c6 | c7 | c8 | c9 | ca | cb |

图 3.1 车的位置价值

Fig.3.1 Position value of rook

如图 1 车在 0xc3 的位置上，也就是棋局初始化时的位置，那么他位置价值是最低的；如果车能威胁到对方的九宫，且在这个位置上可控制的范围较大那么它位置价值就高，如果车在对方的九宫中间，那么对帅的威胁将是致命的。局面评估函数的位置价值的计算如公式（3.2）所示。

$$SEVP = \sum_{i=1}^{i<=\text{走棋方棋子的总数}} POSVALofCHESSMANi \qquad (3.2)$$

其中 POSVALofCHESSMANi 表示第 i 个棋子在棋盘上的位置价值。

### 3.1.3 其它评估因素

中国象棋中车的控制范围最大，但是如果它的四个移动方向都被己方棋子阻挡的情况下它的战斗力无法发挥出来，那么车对棋局的作用就会大打折扣。其它的棋子面



中国知网 https://www.cnki.net

临同样的问题，这个就是棋子的灵活性。灵活性对于能过河攻击对方的兵种来说尤为重要。马的落点与车和炮比相对较少，在这样的情况下如果马腿上有棋子挡住了马的行进路线，所以应尽量让马处在子力疏密程度适中的条件更能发挥它的作用。如果走棋方的某个棋子在对方的攻击范围内，这样的状况对于走棋方而言是不利的应该在局面评估函数中得到体现，应给走棋方减分，与受到对方威胁相对的是受到己方的保护，说明棋子是有“根”的，这种情况应该为走棋方加分，这就是常说的棋子威胁与保护。如果己方的一个车和一个马被对方牵制那么己方的可调动的棋子数量将减少，这对己方是不利的。将帅安全、棋子配合、象棋中的一些特殊的形式等都是局面评估函数应该考虑的[24]。

上文所述的局面相关的因素都是大家普遍公认的，还有很多因素不为人们所知，或者即使了解也无法对其量化，由此可见一个好的人机对弈程序的设计不仅需要程序设计人员的努力还需要棋艺高超的棋手作为顾问，因为博弈程序对于具体的棋类知识有很强的依赖性。

## 3.2 局面评估函数与博弈系统的关系

评估函数使计算机了解当前局面的状况，如果局面评估函数中的包含的棋类知识量达到一个人类高手的水平且被正确的量化到局面评估函数中，那么局面评估函数对棋局状况的描述是客观准确的。博弈系统中另一个重要因素是博弈树搜索,这两个方面都是非常耗费时间的，如果局面评估函数中具备的知识越多，它给出局面评估值的速度就越慢，而博弈树搜索随着搜索层数的增加会耗费更多的时间。中国象棋的博弈系统有两种类型，一种被叫做知识型，将时间花费在局面评估函数上用来弥补博弈树搜索的不足被叫做知识型；另一种是将时间花费在博弈树搜索上，采用相对简洁的局面评估函数，使得博弈树搜索的层数尽可能的多，以速度来获取胜利被叫做速度型。速度与知识之间的关系如图 3.2 所示。由图 3.2 可知，知识量和速度是一对成反比的量，而程序的棋力则是二者之积，即Performance（性能）=Speed（速度）×Knowledge（知识），所以单纯追求知识量的增加把时间花费在局面评估上或者是把时间都用在博弈树搜索上对于提升博弈系统的整体性能是不可取的[25]。博弈系统设计者的任务之一就是找到速度与知识量的平衡点，使得博弈系统的整体性能达到最佳。



中国知网 https://www.cnki.net

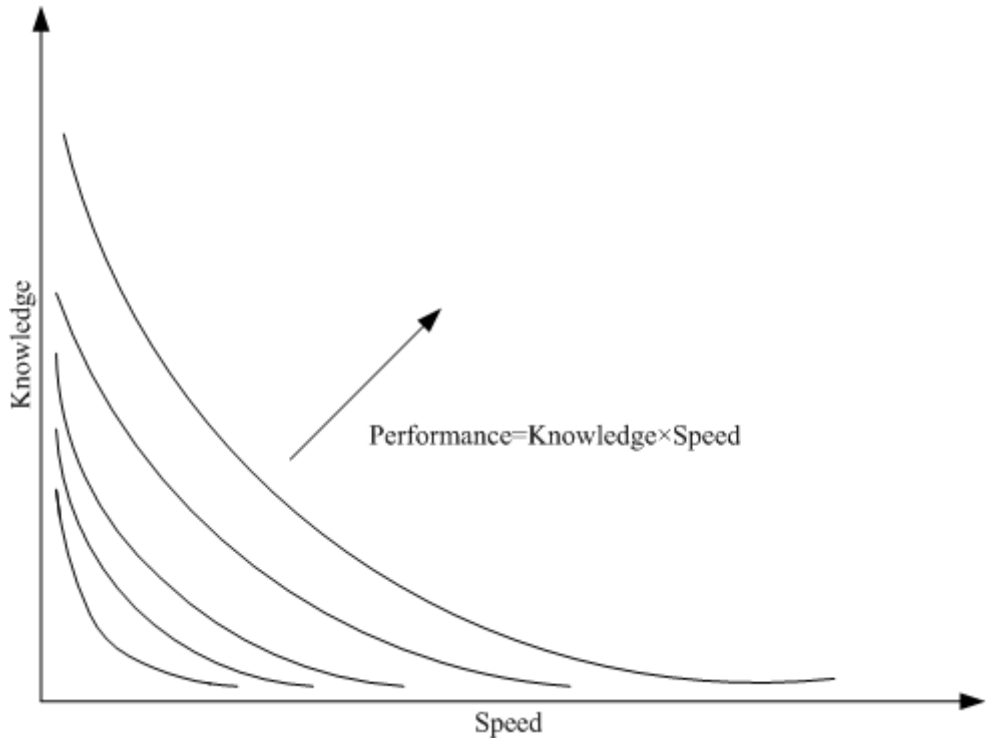



图 3.2 速度与知识之间的关系
Fig.3.2 The relationship about speed and knowledge



中国知网 https://www.cnki.net

# 第四章 博弈树搜索算法研究

## 4.1 博弈树搜索

### 4.1.1 搜索

搜索在人工智能中是一个基本问题，是推理中不可或缺的一部分，它直接影响到人工智能系统的运行效率和工作性能，因此有些科学家把它列为人工智能研究的四个核心问题之一[26]。在人工智能的发展过程中对搜索技术做了大量的研究工作，也取得了大量成果，搜索效率的提高和搜索复杂性理论的研究是正在开展的关于搜索的研究。人工智能领域中要面对的问题多数是非结构化的或者是结构不良的问题，对于这两类问题一般不存在成熟的求解方法供人们利用，只能根据具体问题用已有的知识一步一步的搜索推进[27]。在搜索前进的过程中存在着如何有效寻找可用知识的问题。此外，很多问题存在多条解决问题的路径，这将引出选择哪一条路线进行求解来获得较高的系统执行效率的问题。类似于上述情况根据问题的具体情况在搜索中不断寻求可利用的知识而构造一条代价较小的路线，使问题得到解决的过程就是搜索。

人工智能领域中即使是结构性良好，理论上已有现成的算法可以利用的问题，也可能会因为问题本身的复杂性和计算机在时间和空间上的局限性而不得不通过搜索来求解问题。计算机博弈就是一个典型的例子，计算机为了取得胜利，需要在走棋之前考虑到所有可能的走法，从所有的走法当中选择一个作为应对策略，找到这样一个走法的算法并不难，但是计算机在搜索过程中的时间和空间消耗却是巨大且无法承受的[28]。

搜索分为盲目搜索和启发式搜索两种，启发式搜索中加入了与具体问题相关的启发式信息，用来修正搜索的前进方向，加快找到最优解的速度，盲目搜索是按既定的策略进行搜索，搜索过程是按预先设定的路线前进，没有将问题本身的特性应用到搜索中，这种方式效率不高，不适于用来解决复杂问题。从搜索效率上看，启发式搜索明显优于盲目搜索，但是启发式搜索需要与具体问题相关的知识信息提供帮助，而启



中国知网 https://www.cnki.net

发信息并不是容易找到的，所以盲目搜索的应用还是很多。

### 4.1.2 中国象棋博弈树

中国象棋博弈树是由对弈双方共同构成的。在初始棋局状态下红方先走，他会考虑在初始情况下所有可行的走法，对所有的行动做评估，在这个行动集合当中找出一个对自己最有利的走法，并采取行动使棋局状态发生改变，红方的第一步棋完成，轮到黑方走棋，他也根据当前的棋局状态得到所有可行的走法构成一个行动集合，对所有的行动做评估，选择一个对自己最有利同时对红方却是最不利的走法，黑方采取行动，使局面发生改变[29]。在双方博弈的过程中将棋局状态理解为博弈树的节点，走法理解为博弈树的分枝。图 4.1 就是一棵博弈树。

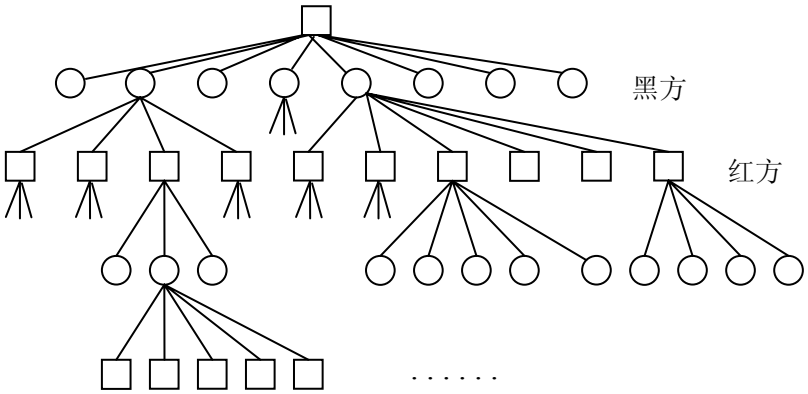



图 4.1 博弈树

Fig.4.1 Game tree

如果将中国象棋从初始棋局状态把每一个中间节点所有可行的走法都列出来，直到分出胜负的叶子节点为止，这样构成一棵包含了所有局面的树，叫完全博弈树。现实中的每一盘棋只是完全博弈树的一条从根节点走到叶子节点的路径而已。如果计算机能够建立一颗完全博弈树，那么无论对手如何走棋，计算机总能找到最佳的应对策略保证其胜利。中国象棋的每个棋局状态下的走法数目在 20～60 之间，按 40 来计算的话，每个棋局状态下都有 40 个分枝，假设双方各走五十步能分出结果，那么生成的节点数是 $40^{100}$个，可以推出 $40^{100}=(2^{20})^{10}\times10^{100}\geqq10^{160}$，这个数字远超出现在计算机的处理能力，所以构造一棵完全博弈树是不现实的，计算机博弈只能采取别的解决方案。



中国知网 https://www.cnki.net

可行的做法是在有限的时间内，以当前所处的棋局状态为初始状态生成一棵有限层数的博弈树，在叶节点上调用局面评估函数对叶节点进行评估（这里所说的叶节点可能不是分出胜负的节点而是完全博弈树的中间节点），得到对计算机最有利的走法来采取行动，如同人类棋手一样，如果计算机在有限的时间内展开的层数越多证明它看的越远，局面评估函数越接近实际的局面状况则说明计算机对棋局状态的理解越准确，如果计算机在这两项的综合能力超过人类的话那么它的棋力就在人类之上。本章描述的内容使计算机尽可能的增加搜索层数，这样计算机才能把握住棋局走势。

## 4.2 基本的搜索算法

博弈树搜索算法必须解决三个基本问题：首先行动的产生，博弈的一方要在他所处的棋局状态下，找到行动集合，然后在这个行动集合中找到最佳走法，问题在于这个行动集合中的行动在象棋中也就是走法执行后的棋局状态，应该逐个产生，然后产生一个计算一个，还是将他们一次都生成，然后再计算？这个问题取决于问题的复杂度，对于较为简单的情况可以全部生成，这样可以得到更多的信息，但是对于比较复杂的情况来说（两个国家在战争中的后勤调度）应该每次生成其中的一部分或是逐个生成。第二个问题是搜索方式的选取，应该先在内存中生成一棵完整的树再进行搜索还是仅仅将要搜索的节点放到内存中。这是一个比较容易的解决，当然是只将要搜索的节点放到内存中。只要是棋类稍微复杂一点，它所产生的节点时如果想全部保存的内存中都是无法做到的。第三个问题是搜索顺序如何选择，是选择深度优先原则还是广度优先原则，当然它们前提都是在有限深度的情况下进行的。从有限深度的博弈搜索来看，一个节点得到估值的情况有两种，一种是是它搜索的层数达到了预先指定的搜索层数，然后局面评估函数根据它所考虑的因素执行算法对格局进行评估，返回对局面的评估值，另一种是在没有达到预先指定的搜索层数时，它已经得到了问题的最优解（比如已经得到了杀棋）。综合分析以上情况，应该采用深度优先策略来对博弈树进行的搜索。本节介绍的搜索算法都是基于以上原则运行的。

### 4.2.1 极大极小值算法

博弈树是由对抗性的两方来共同决定的。假设局面评估函数的分值越大代表对红方越有利，那么红方在选择走法时会去选择局面评估函数返回的分值最大的走法，对于黑方来说就会采取钳制措施，选择局面评估函数返回的分值最小的走法作为应对策



中国知网 https://www.cnki.net

略[30]。需要说明的是在这个过程中计算机会认为博弈的双方都足够聪明，会选择对自己最有利的走法而不会去冒险。基于上述想法，冯.诺依曼提出了极大极小值算法图4.2 表示的是由搜索算法展开的层数为 4 的博弈树，其中方块代表红方当前面临的棋局状态，圆圈代表黑方当前面临的棋局状态，搜索是深度优先的所以先到达叶子节点再逐层倒推，在博弈树的叶子节点上调用局面评估函数，将局面评估值返回给上一层，如果上层是方块代表红方，他将选择对自己最有利的走法也就是局面评估值最高的节点，以D节点为例它的取值可表示为：valueof(D)=max(valueof(H),valueof(I))，节点D的取值为 10，节点E的计算方式同D，得到的值为 13。B节点在计算取值时，由于它属于黑方的节点所以它的取值计算方式为：valueof(B)=min(valueof(D),valueof(E))，其值为 10。其它所有节点都可按这种方式得到值，在图 4.2 中极大极小值算法红方最后选择的走法将局面从A转换到C的走法。

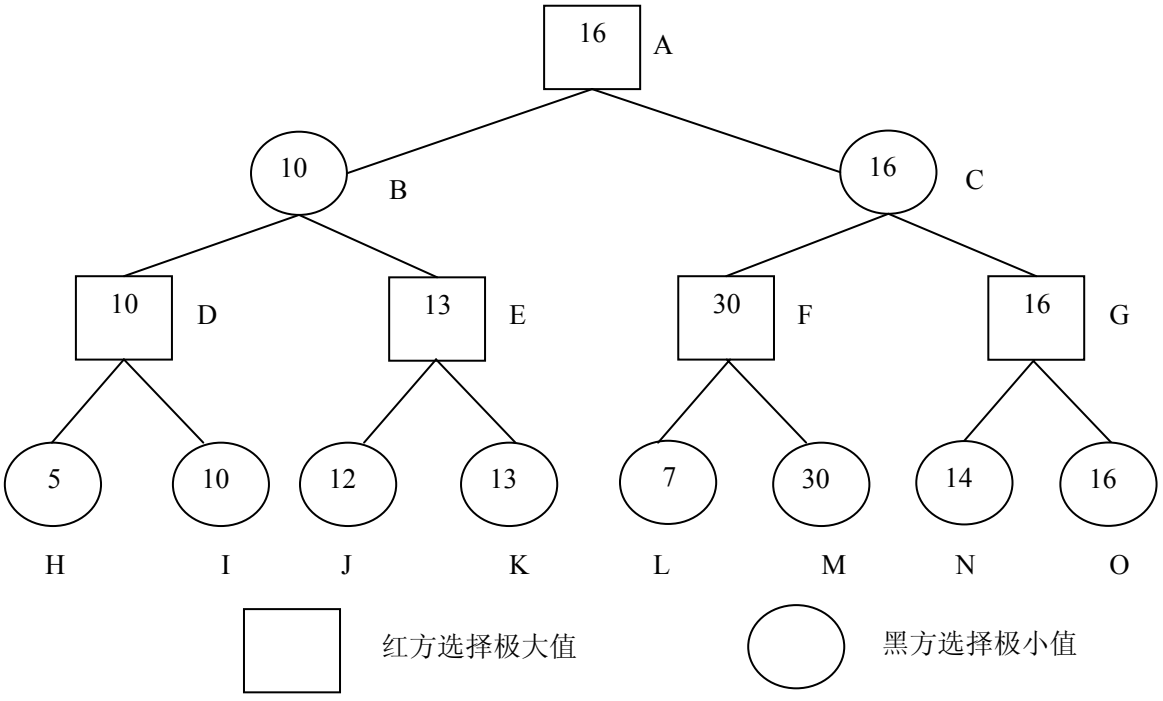



图 4.2 极大极小值算法树
Fig.4.2 Minimax algorithm tree

极大极小值算法的伪代码如下：

```
int    MiniMax(position p,int depth)
{
    int    bestvalue,value
```



中国知网 https://www.cnki.net

```
if(Gameove) //检查棋局是否结束
    return evaluation(p);//棋局结束，返回估值
if (depth<=0) //如果是叶子节点，返回估值
    return evaluation(p);
//如果是红方，让初始的最佳值为最小，这里假设红方取局面评估函数//的最
//大值取最小值道理是一样的。
if(p.color== red)
    bestvalue =-INFINITY;
else if (p.color==black)
    bestvalue =INFNITY;//不是，让初始最佳值为最大
for(each possible move m)//试探每个走法
{
    MakeMove(m);//执行走法改变棋局状态
     value=MiniMax (p,d-1);//递归调用MiniMax向下搜索
     UnMakeMove(m);//回滚到之前的状态
     if(p.color==red)
          bestvalue=max (value,bestvalue);//红方取最大值
     eles if (p.color==black)
          bestvalue ==min(value，bestvalue); }//黑方取最小值
    return bestvalue;} //返回最大/最小值
```

从以上伪代码可以看出，极大极小值算法对于博弈的双方是非常敏感的，对于不同的走法方它的处理方式是不同的，其形式很容易理解，却略显繁琐。Knuth 和 Moore 在 1975 年提出了在形式上的改进，消除了对于双方的区别对待，使得算法的形式更简洁。这个算法要求博弈双方都取最大值，其伪代码如下：

```
int NegaMax(position p,int depth)
{
    int n,value,bestvalue=-INFINITY;//最大值初始为负无穷
    if(game over(p))//游戏结束，胜负已定
    {
        return evaluation(p); }//返回估值
```



中国知网 https://www.cnki.net

```
        if(depth==0)
    {
        return evaluation(p); }
//达到指定的搜索深度，返回局面评估值
    for( each possibly move m)//试探每个走法
    {
        Make move(m);//执行走法改变棋局状态
        value=- NegaMax(p,depth-1);//递归向下搜索
        Unmalemove(m);//回滚到之前的状态
        if(value>=bestvalue)
        {
            bestvalue=value; }}//取最大值
        return bestvalue; }//返回最大值
```

从效率上极大极小值算法和负极大值算法是等价的，二者都是对当前的棋局状态做一个有限层数的完全搜索。从代码的形式上看，负极大值算法要简洁的多，但是对于理解该算法却是更加困难了，得到好处是应用更方便，今天大多数博弈程序采用的是基于负极大值形式的搜索算法。这个算法要正确的运行还需要对局面评估函数的形式稍加改动，这时的局面评估函数对走棋方必须敏感，如果对于红方走棋返回的是正的估值那么黑方走棋局面应该返回负的估值。

### 4.2.2 α-β 剪枝算法

在 4.2.1 节中论述的极大极小值算法和负极大值算法，它们是对有限层次做完全搜索的算法，结合图 4.2 分析一下这两种算法的执行效率，图 4.2 表示红方走了两步棋而黑方走一步棋的一个博弈树，这里叶子节点以外的节点都需要生成该局面下的所有走法，也就是生成所有的分枝，然后试探所有的分枝得到新的节点，一直到达叶子节点返回局面评估函数给出的分值，逐层倒推，得到分值最高的走法。这个过程中，非叶子节点都要调用走法生成模块开生成该棋局状态下的所有走法，对于叶子节点要调用局面评估函数来返回局面的估值，如果搜索深度为 2（根节点算作第一层）那么将调用一次走法生成模块，调用 40 次局面评估函数。如果搜索深度为 5 层，需要调用走法生成模块的节点数目为：$40+40^2+40^3=65640$ 个需要调用局面评估函数的节点数



中国知网 https://www.cnki.net

目为：$40^4$=2560000 个。根据第二章的介绍，每次调用走法生成模块都会生成该棋局状态下的所有可行走法，即使数据结构设计的再合理也会带来大量的判断和操作。根据第三章的介绍如果想使得计算机对棋局的判断准确就要加入更多的知识，同样意味着更长的计算时间，博弈系统中的这两个部分都非常耗费时间，它们都是协助搜索算法完成工作的重要组成部分。只是展开了双方各走走两步棋的局面就产生了这样的计算量，随着搜索层数的增加处理的节点数呈指数增长，利用负极大值算法搜索博弈树使计算机搜索的层数达到具备实战能力的水平是不可能的。

现在搜索算法的瓶颈是要搜索更多层数使得计算机看到的局面尽可能的接近对弈的终局和所花费时间之间的矛盾。根据实际情况有些节点和这个节点下的分枝是不需要做处理的。如图 4.3 所示。

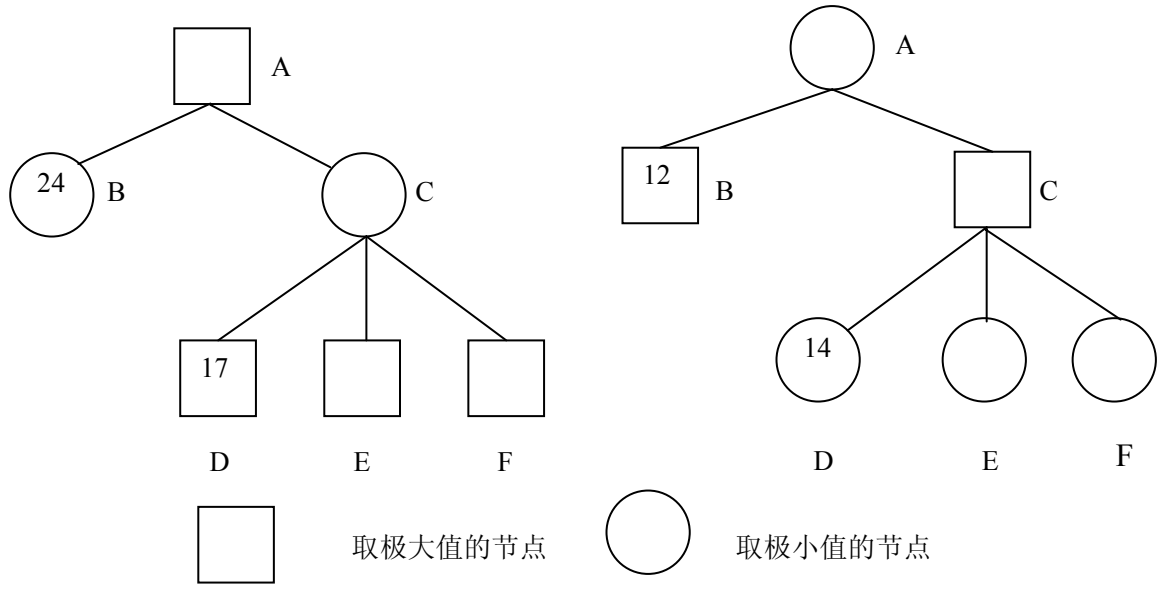



图 4.3 α-β 剪枝原理
Fig.4.3 α-β cutoff theory

如图 4.3 右图所示的情况，B 节点的评估值为 24，C 的子节点 D 的评估值为 17，由于 C 是取极小值的节点，故该节点的评估值计算方式为：valueof(C)=min(valueof(D), valueof( E), valueof(F))，因为 D 节点的评估值是 17 所以 C 节点的评估值不会大于 17，再看 A 节点，A 节点属于取极大值的节点，其评估值的计算方式为：valueof(A)=max(valueof(A), valueof(B))，因为 D 节点的评估值为 17，valueof(B)≦ 17，无论 D 节点兄弟节点的评估值是多少 C 节点都不会被 A 选中作为扩展的分枝，因此对于



中国知网 https://www.cnki.net

节点 D 兄弟节点的分析是没有必要的，应该将它们剪掉以节省计算机的开销，诸如这样的情况被叫做α剪枝。如图 4.3 左图所示的情况，B 节点的评估值为 12，C 的子节点 D 的评估值为 14，由于 C 是取极大值的节点，故该节点的评估值计算方式为：valueof(C)=max(valueof(D), valueof( E), valueof(F))，因为 D 节点的评估值是 14 所以 C 节点的评估值不会小于 14，再看 A 节点，A 节点属于取极小值的节点，其评估值的计算方式为：valueof(A)=min(valueof(A), valueof(B))，因为 D 节点的评估值为 14，valueof(B)≧14，无论 D 节点的兄弟节点的是多少 C 节点都不会被 A 选中作为扩展的分枝，因此对于节点 D 的兄弟节点的分析也是没有必要的，应该将它们剪掉以节省计算机的开销，诸如这样的情况被叫做β剪枝。根据以上的理论，Brudno 在 1963 年在他的论文《Bounds and Valuations for Abrulging the Search of Estimates,problems of Cybernetics》提出了 α-β 剪枝算法，1975 年 Knuth 和 Moore 给出了 α-β 剪枝算法的数学正确性证明。α-β 剪枝算法的伪代码如下：

```
int AlphaBeta(depth,alpha,beta)
{
    if (gameover)//分出胜负
    {
        return evaluation();}//返回局面评估值
    if (depth==0)
    {
        return evaluation();//到达叶子节点返回局面评估值}
        if (is min node)//取极小值节点的处理方式
    {
        for (each possibly move m )//试探每个走法
        {
            make move m;//执行走法，修改棋局状态
            score=AlphaBeta(depth-1,alpha,beta);//递归搜索子节点
            unmake move m;//撤销走法回滚得到上一层
            if (score<beta)
            {
                beta=score;//取极小值
```



中国知网 https://www.cnki.net

```
            if(alpha>=beta)
                return alpha;//alpha剪枝}}
                        return beta; }//返回极小值
    else
    {//取极大值的节点
        for (each possibly move m )//试探每个走法
        {
            make move m;//虚拟执行走法，修改棋局状态
            score=AlphaBeta(depth-1,alpha,beta);//递归搜索子节点
            unmake move m;//撤销走法回滚到上一个棋局状态
            if (score>alpha)//取极大值
            {
                alpha=score;
                if(alpha>=beta)
                    return beta;//beta剪枝，抛弃兄弟及其后继节点    }}
                return alpha; }}//返回极大值
```

很多资料都做实验分析了α-β剪枝算法的搜索的效率，它可以忽略很多无关大局的节点的搜索。对非叶子节点的剪枝，不仅是这个节点，还包括以它为根的子树上的所有节点都被剪掉，其搜索相同深度搜索的节点数远少于极大极小算法，在相同的时间α-β剪枝算法可以搜索更多的层数，即计算机能够“看的更远”了[31]。

形式上α-β剪枝算法和极大极小算法相似，繁琐、应用不便，在取极大值节点上进行beta剪枝，在取极小值的节点上进行alpha剪枝，可以将负极大值的形式应用在α-β剪枝算法上[32]。这样在任何一层都只进行beta剪枝。其伪代码如下：

```
int alphabeta(int depth，int alpha，int beta)
{
    if(Game over) //胜负已分
        return evaluation();//返回估值
    if(depth==0) //叶子节点
        return evaluation();//调用估值函数，返回估值
```



中国知网 https://www.cnki.net

```
for(each possibly move m)//对每一种可能的行动进行探测
    Make Move(m);// 虚拟执行走法，修改棋局状态
value=-alphabeta(depth-1,-beta,-alpha);//递归搜索子节点
    UnMake Move(m);// 撤销走法回滚到上一个棋局状态
    if(value>=alpha)
        alpha=value;//取最大值
    if(alpha>=beta)
        break; }//beta剪枝
return alpha;}//返回最大值
```

经过几十年的发展出现了很多的改进的博弈树搜索算法，但是到目前为止α-β剪枝算法仍是使用最广泛的博弈树搜索算法。α-β剪枝算法是博弈树搜索的基础技术。

### 4.2.3 α-β 剪枝算法的性能分析

表 4.1 是总结多篇论文和相关资料得到的数据，对于极大极小值算法来说它是对全窗口进行搜索，所以搜索的叶节点数随着搜索深度的增加搜索节点的也以 $40^n$增加，这也符合象棋的平均分支因子为 40 的说法，极大极小的思想为后来的博弈树搜索起到了抛砖引玉的作用，但是它的效率却不理想，搜索为 5 层时，节点数就达到了 $10^8$的数量级，其花费的时间相当长，虽然这样做计算机不会犯错误，但是在时间上是不允许。

α-β 剪枝算法的引入使得剪枝效率大为提高，搜索两层时就体现出它的优势，这使得基于极大极小理论的算法应用在中国象棋博弈树搜索上成为可能。之所以不同的资料对同一算法有着不同的统计数据，除了统计方式的不同，关键在于他们使用的走法生成模块不同，不同的走法生成模块生成走法的顺序不同，会影响博弈树的展开顺序。

表 4.1 极大极小算法与 α-β 剪枝算法搜索节点数比较
Table4.1 Search nodes amount of MiniMax and α-β

| 搜索层数 / 算法 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| MinMax | 45 | 1500±200 | 67000±2000 | 235000±10000 | 100000000±10000000 |
| α-β | 45 | 550±50 | 11000±1500 | 104000±2000 | 1810000±500000 |



中国知网 https://www.cnki.net

α-β 剪枝算法提高了极大极小值算法的效率，我们考虑一下如图 4.4 的情况，来分析影响 α-β 剪枝算法效率的因素。图 4.4 左图，如果 α-β 剪枝算法先搜索到节点 B 然后展开它的子节点 D、E、F，C 节点的值是 30 所以对 D、E、F 的搜索是徒劳的，然而 α-β 搜索算法先展开的是 B 节点所以它无法知道后来会有更好的策略，这是对于 α 剪枝所产生的 α-β 剪枝算法失效的例子。观察图 4.4 右图可知对于 E、F 的搜索亦是不必要的，这是对于 β 遇到的 α-β 剪枝算法失效的例子。

由上述分析可知α-β剪枝算法对于节点的展开顺序非常敏感，这只是原理分析。Knuth和Moore的研究对此做的工作得到了大家的公认，东北大学人工智能实验室在他们研究的基础之上做了更深入的研究，后者的研究表明在最理想情况下(极小树), 其生成的节点数目如公式（4.1）、（4.2）所示[34]。

$$N_{depth}=2W^{depth/2}-1 \quad (\text{depth为偶数}) \tag{4.1}$$

$$N_{depth}=W^{(depth+1)/2}+W^{(depth-1)/2} \quad (\text{depth为奇数}) \tag{4.2}$$

其中depth为搜索深度，W为分枝因子。在使用极大极小算法时，需要搜索的节点数是$N_{depth}=W^{depth}$，即深度为depth的极大树，如果α-β剪枝算法能发挥到最佳状态其搜索深度为depth的节点数相当于搜索深度为depth/2 的不做任何剪枝的节点数[34]。分析博弈树可得，如果博弈树的最左路的分枝就是最佳路径，也就是理智的双方最为精彩的对弈走法序列，那么就可以最大限度的将右路各分枝陆续剪掉，从而使搜索的节点数仅是深度为depth的极大树的$2\sqrt{W^{depth}}$ [35]。

虽然α-β剪枝算法使得博弈树搜索的节点数目极大的降低了，但是这对于计算机对战人类高手还是远远不够的。众多研究人员加入到改进α-β搜索算法的行列中，产生了很多改进技术。将α-β剪枝算法的参数alpha和beta视为窗口[36]，基于这种思路产生了Fail-Soft alpha-beta算法、渴望搜索算法、极小窗口搜索算法（Minimal Window Search/PVS）、宽容搜寻算法（Tolerance Search）等；α-β剪枝算法对节点排序的敏感，通过排序优先搜索最佳走法从而优化算法性能，方法有吃子走法、杀手走法（Killer heuristic）、类比法（Method of analogies）和历史表法（Historic heuristic）等[37]。搜索过程中遇到已经搜索过的局面可以利用之前的知识对其进行评估给出走法和分值，这种节省搜索时间的方法可以有效提高搜索效率被叫做置换表法[38-40]；对于同一个搜索算法，在时间相同的情况下在棋局发展的不同阶段搜索的层数差别很大，针对这种



中国知网 https://www.cnki.net

情况提出了迭代加深的策略[41-42]。还有开局库，残局库等等[43-46]。

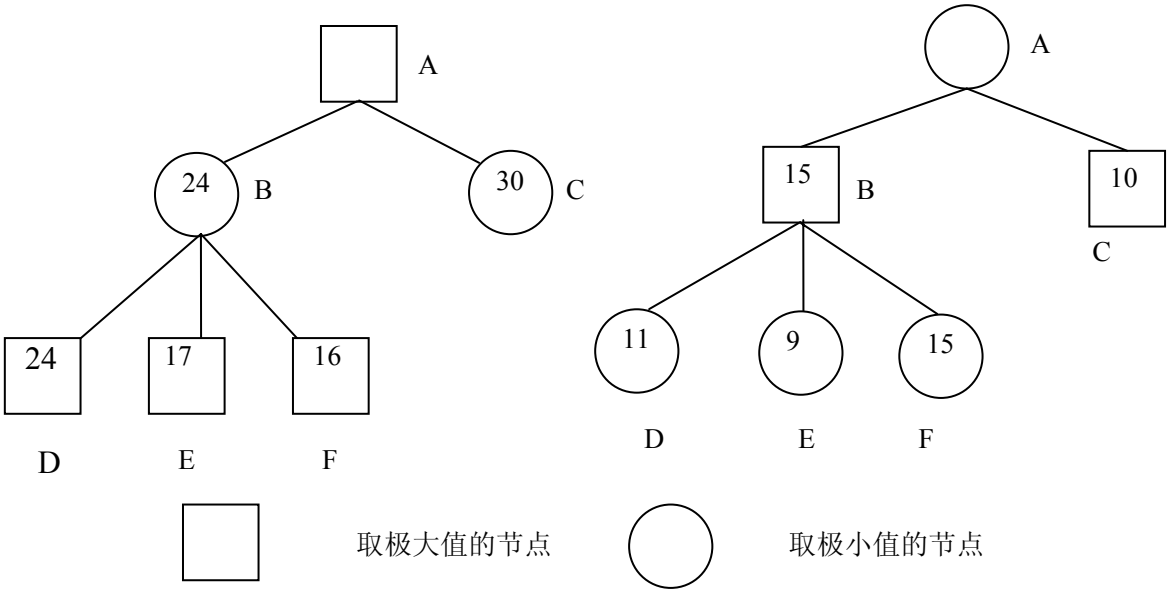



图 4.4 α-β 剪枝算法
Fig.4.4 α-β pruning algorithm

## 4.3 基于窗口原则的改进策略

### 4.3.1 Fail-Soft alpha-beta 算法

在使用α-β剪枝算法时将参数alpha和beta分别赋值为-∞和+∞，递归的过程中alpha和beta的值不断被修改。伴随着搜索的进行，alpha和beta限定的范围会越来越小，落在这个范围之外的节点会越来越多。算法的剪枝效率也越来越高[47-49]。如果算法开始执行时就将alpha和beta构成的搜索范围缩小，那么搜索过程会有更多的剪枝，也就是剪枝的效率会进一步提高，但是这样做会有三种不可预料的结果：

（1）目标值在 alpha 和 beta 之间。

（2）目标值大于 beta。

（3）目标值小于 alpha。

在（2）、（3）两种情况下要重新搜索，之前计算机所做的工作没有得到利用，α-β 剪枝算法返回的是 alpha 的值，所以无法返回比 alpha 小的值。对基本的 α-β 剪枝算法做修改，使它能够返回当前的分值，这样的 α-β 剪枝算法被称为 Fail-soft alpha-beta 算法。算法伪代码如下：



中国知网 https://www.cnki.net

```
int FailSoftalphabeta(int depth,int alpha,int beta)
{
    int cur=-INFINITY;//保存与搜索结果相关的信息
    if(Game over) //胜负已分
        return evaluation();//返回估值
    if(depth==0) //叶子节点
        return evaluation();//调用估值函数，返回估值
    for(each possibly move m)//对每一种可能的行动进行探测
        Make Move(m);// 虚拟执行走法，修改棋局状态
 value=- FailSoftalphabeta (depth-1,-beta,-alpha);//递归搜索子节点
    UnMake Move(m);// 撤销走法回滚到上一个棋局状态
    if(score>cur)
{
cur=score;
    if(value>=alpha)
        alpha=value;//取最大值
    if(alpha>=beta)
        break;//beta剪枝
}
return cur; }//返回边界或者最佳值
```

观察伪代码可知若该算法返回的值 cur 小于初始的窗口说明我们要找的值小于alpha，若该算法的返回值 cur 大于初始的窗口则说明我们要找的值大于 beta。源代码和实验均表明该算法并未提高α-β剪枝算法的剪枝效率，却给很多基于窗口原理的α-β剪枝算法改进提供了方便。

### 4.3.2 渴望搜索

渴望搜索（aspiration search）是对Fail-Soft alpha-beta算法的改进，在Fail-Soft alpha-beta算法中alpha和beta值分别是-∞和+∞，在这样一个大的范围伴随搜索不断缩小由alpha和beta设置的范围，如果初始搜索范围可以缩小就能加速算法的收敛速度[50-51]。算法实现面临三个问题要解决：



中国知网 https://www.cnki.net

（1）如何确定窗口的基准值。在 Fail-Soft alpha-beta 算法中的 alpha 和 beta 值分别是-∞和+∞，可以认为基准值是 0。

（2）如何确定窗口的大小。也就是初始时搜索范围。在 Fail-Soft alpha-beta 算法中的窗口大小是从-∞到+∞。

（3）如何处理因为缩小初始搜索范围带来的窗口边界溢出问题。在 Fail-Soft alpha-beta 算法中的窗口大小是从-∞到+∞不会产生溢出。

通过 d-1 层的全窗口搜索，将返回值作为 d 层渴望搜索窗口的基准值，在一般情况下进行深度为 d-1 层的搜索所需要的时间仅是搜索深度为 d 层的 1/15 到 1/5。窗口的大小要根据局面评估函数中参数的数量级来给定，由于不同的局面评估函数的参数数量级是不同的，所以没办法统一，国际象棋的经验是窗口的宽度为一个兵的价值。如果搜索得到的返回值小于设定的窗口的下限则表明要找的值在-∞到 alpha 之间，需要将窗口修改为-∞到 alpha 进行搜索；如果搜索得到的返回值大于设定的窗口的上限则表明要找的值在 beta 到+∞之间，需要将窗口修改为 beta 到+∞进行搜索。采用以上解决方案的搜索算法叫渴望搜索。其伪代码如下：

```
int AsPirationsearch(int dePth)
{      // INFINITY代表一个大数
    int x=FailSoftalphabeta (depth-1,-INFINITY,INFINITY);
    int alpha=X-WINDOW;//设定窗口左界
    int beta=X+WINDOW;//设定窗口右界
    int cur=FailSoftalphabeta(depth,alpha,beta);//进行搜索
    if(cur<alpha)
//处理估值偏高情况
        cur=FailSoftalphabeta(depth,-INFINITY,alpha);
    if(cur>beta)
//处理估值偏低情况
        cur=FailSoftalphabeta(depth,beta,INFINITY);
    return cur;}//返回估值
```

分析渴望搜索算法，当搜索层数是一、二时，渴望搜索算法将会访问更多的节点，这必将增加计算机的处理时间。原因在于，渴望搜索算法需要用到上一层的完全搜索



中国知网 https://www.cnki.net

来作为它的基准值，所以实际分析的节点数目应该是两层之和。但是到了第三层，它的剪枝优势就得以体现，这是因为此时 d-1 层和 d 层所花的时间比为 1:15~1:5，就是用 1 的额外开销获得了 5 到 15 的优化。

### 4.3.3 极小窗口搜索

极小窗口搜索（Minimal Window Search）也被叫做 PVS（Principal Variation Search）其原理是：假设当前的走法是最好的，其后继不如它的前驱，直到有节点被证明是最佳的为止。对于中间节点来说，对它的第一个分支做全窗口搜索后得到一个值 value，后续的分支就以（value, value+1）这样的小窗口进行搜索，这个算法的本意是想以零窗口建立极小树以达到提高效率的目的。算法基于后续节点的评估值和它的前驱节点的评估值之间的变化微小这样的猜想。执行过程为：

（1）对于第一个节点，按照原来的范围（-∞,+∞）进行搜索，得到一个最优解 value。

（2）用（value,value+1）作为窗口进行测试。

（3）如果得到的值大于 value 并且小于 beta 时，表明有更好的方法，需要对(value,beta)进行探测。

（4）否则，再判断得到的值 newvalue 大于 value，表明 newvalue 是一个更好的行动，应该用它来代替原来的 value。

（5）如得到的值 newvalue 小于 value，表明这个走法还不如之前的走法，不必再分析。

下面给出伪代码的实现：

```
int PrinciPalVariation(int depth,int alpha,int beta)
{
    if(Gameover)
        return evaluation();//胜负已分，返回估值
    if(depth==0)
        return eveluation();//叶子节点返回局面评估值
    MakeMove(m);//生成第一个分支
    best=-PrinciPalVariation(depth-l,-beta,-alpha)//全窗口搜索
        UnMakeMove(m);//撤销走法
```



中国知网 https://www.cnki.net

```
for(each possibly move m)//从第二个节点对每一种可能的行动进行探测
{
    if(best<beta)//是否不能进行beta剪枝
    {
        MakeMove(m);//虚拟采取这个行动
value=-Prineipalvariationn(depth-l,-alpha-l,-alpha);//极小窗口测试
        if(value>alpha&&value<beta)//估值过低，重新完整估值
            best=-PrineiPalVariation(depth-l,-beta,-value);
        else if(value>best)//命中
            best=value;
        unMake Move(m); }}//撤销走法
            return best;}//返回最佳值
```

许多研究者认为极小窗口搜索在效率上优于 α-β 剪枝搜索算法，实践也证实了这一点，这使得 PVS 在实际的应用比较普遍，为博弈程序设计者所熟知。这个算法还有一个名字叫 NegaScout 搜索算法，但是他的设计者 A.Reinefeld 在其主页上公布源代码时出现严重错误，使得算法的剪枝效率严重下降。

### 4.3.4 基于窗口原则搜索算法小结

表 4.2 α-β 剪枝、渴望搜索和极小窗口搜索节点数比较
Table4.2 Search node amount of alpha-beta, Aspiration and PVS

| 搜索层数 / 算法 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| alpha-beta | 45 | 550±50 | 11000±1500 | 104000±2000 | 1810000±500000 |
| 渴望搜索 | 73±3 | 680±100 | 1100±200 | 42000±1000 | 940000±20000 |
| 极小窗口 | 48±3 | 550±50 | 8400±200 | 74000±2000 | 700000±100000 |

在表 4.2 中列出了 α-β 剪枝、渴望搜索和极小窗口搜索在 1～5 层搜索的节点数比较，该表是总结相关资料得到的数据，Fail-Soft alpha-beta 搜索算法仅仅是在形式上对 α-β 剪枝算法做了修改，在性能上并没有产生影响，这点只要对比二者的源代码便可得知，而且很多人认为 α-β 剪枝算法就是指 Fail-Soft alpha-beta 搜索算法。4 层以内渴望搜索的效率高于极小窗口搜索但是在 5 层以后极小窗口剪枝的效率却高于渴望搜索。对渴望搜索和极小窗口搜索的算法源代码分析可知：基准值源于同层搜索，这



中国知网 https://www.cnki.net

样比渴望搜索要准确，尤其是出现局面剧烈变化时；极小窗口搜索的剪枝效率在浅层不如渴望搜索但是随着搜索层次的加深其效率将高于后者。上一节的描述表明这些基于 α-β 剪枝算法的改进算法都会受节点展开顺序的影响，使搜索算法的效率具有不确定性，剪枝效率也大打折扣，下一节将介绍一些优化节点排序的方法，这些方法使博弈树搜索算法的剪枝效率得到了提高。

## 4.4 博弈树节点排序

博弈树节点展开顺序对于搜索算法的影响我本章就不赘述了，若想让搜索算法发挥其巨大潜力就要对博弈树节点的展开顺序做调整，也就是对生成的走法序列根据某种规则做调整，使得搜索算法先搜索好的走法，这种优化方式叫狭义的启发。如果走法的排序不依赖于搜索结果，仅在得到某个局面下的走法集合后就能对走法进行进行排序，这种方式叫做静态启发，吃子启发就具备这个特性；还有的走法排序方法依赖于搜索以前搜索结果，历史表启发就属于这一类。

### 4.4.1 吃子走法启发

（1）MVV/LVA

用棋力最小的子攻击对手棋力最大的子（most valuable victim/least valuable attacker）该排序方法的思想是用最小的代价换取最大的利益，适用于对吃子的走法的排序。MVV 表示被吃子的值越大，越应该被优先搜索，如果能用一个兵吃掉对方的炮，那么这个走法将被视为是好的。LVA 表示执行攻击的攻击者价值越小越好，如果用兵和马都可以吃掉对方的车，那么用兵吃掉对方的车将会被优先搜索。该排序方法要综合考虑代价和收益，两个方面的因素结合起来对吃子的走法做排序。一般会在平静搜索中应用这种算法，博弈程序一般会应用多种方式对搜索算法进行优化，可能会包括置换表、历史表、杀手列表等等，但是这些动态的启发方法在搜索的开始阶段并没有发挥作用，这时将会体现 MVV/LVA 排序的价值。应用该方法在实现上要求有仅生成吃子走法的走法生成器来配合。系统中的实现策略在第五章中详细介绍，这种方法综合了攻守双方的因素。

（2）对子评价（Static Exchange Evaluation）

SEE 其汉语意思吃子交换的评价，是计算更加复杂的吃子走法排序，可以剪掉一些价值不高的吃子走法。在一个受双方多个棋子控制的位置，双方对这一位置进行争



中国知网 https://www.cnki.net

夺，会依次使用代价最小的棋子来控制这个位置，直到双方在这一位置上的战斗力耗尽无法在投入新的兵力为止，然后统计交换的最终结果，如果对主攻方有利，就优先搜索，否则要放弃这个位置的争夺，因为这样做的代价太大得不偿失。对子在中国象棋中尤为复杂，假设因为炮打隔山子而不能作用到某个位置上，但是由于发生了子力交换使得这个位置在炮的控制下，类似的情况还有蹩马脚等等。正是这些复杂的规则使中国象棋的棋局判断更为复杂。

### 4.4.2 杀手启发

杀手启发是特殊的历史启发，它不依赖于人类的知识，具有普遍性。杀手走法启发简单讲就是搜索某个节点时$A_1$作为第一个被执行的走法，由A1 的后续走法B1 产生截断，回到A1 的父节点搜索A1 的兄弟节点A2 如果B1 是该局面下的合法走法那么B1 很可能会再次产生截断。在博弈树搜索过程中，有很多走法一经搜索就会引发大量的剪枝，而且这些走法通常可能是同一走法，这样在搜索过程中记录下每一层引发剪枝数最多的走法。当下一次搜索到同样的深度时，如果这个走法是该局面下的一个合法走法时，优先杀手走法搜索。对于搜索树的一个中间节点来说，如果它的某个走法引发了剪枝，由于局面的相似性，这个走法也很可能引发其兄弟节点的剪枝。在杀手启发的应用上，多数博弈程序的做法是：为每一层保存两个杀手走法，把产生截断的走法保存到杀手列表中，且优先搜索杀手列表的最新走法，然后搜索次新的走法。在某个层次中如果有新的杀手走法要保存时，将杀手列表的最新走法保存到次新走法的存储空间中，将最新杀手走法保存到最新走法的存储空间中。其代码表示如下：

```
//mv 表示新的杀手走法，如果它不是杀手列表的最新走法就更新。
if (mvKillers[depth][0] != mv){ //mvKillers表示杀手列表。
    mvKillers[depth][1] = lpmvKillers[0];
    mvKillers[0] = mv;
```

杀手列表中的走法不是在当前局面下生成的走法，其合法性并未得到验证，应用时要确保它是合法的走法。

### 4.4.3 历史表启发

吃子走法启发可以对吃子走法做一个排序，使得博弈树展开过程中优先搜索收益大代价小的走法。杀手启发为每一层保存了杀手走法，但是数量有限，然而象棋的分



中国知网 https://www.cnki.net

支因子的平均值在 40 左右，不是每个走法都会吃子，剩余走法该如何排序，如果仅仅是随机排序对剪枝算法的效率提高不利，历史表启发可以解决这一问题。一些以前搜索过的认为是好的走法，在相似局面的情况下仍有可能是好的走法，而且，如果某个走法被多次认为是好的走法那么它极有可能就是最佳走法。如果在后续的搜索过程中，对搜索过的好的走法进行记录和统计，将记录集中分值高的节点先搜索就会增加剪枝的效率。经验证明这样做好于随机排序。应用历史表启发必须解决两个问题：

（1）什么样的节点有资格存储到走法的记录中。

（2）如何给被记录的走法打分。

那些引发剪枝和虽然未引发剪枝但是兄弟节点中的最佳走法，这两类走法都应该被保存到历史表中。对应该添加到历史表中的走法的评分原则，有两种截然相反的看法，一种是认为靠近根节点的走法应该赋予高分而先搜索，另一种是认为远离根节点的走法应赋予高分而先搜索，多数研究人员认为靠近根结的走法应该被赋予高分，因为这样的走法考虑了更多的子节点，考虑的情况相对全面，而远离根节点的走法考虑的情况会相对片面，所以应该将靠近根节点的走法赋予高分。

根据走法所在的层次赋予分值，根据国际象棋所得到的经验，为应保存在历史表中的走法赋予的分值是它所在搜索层数的平方，也就是如果一个走法是n层的最佳走法那么给它赋予的分数是$n^2$，如果在n-2 层还需要对这个走法保存到历史表中，这个走法在历史表中的分值是$n^2+(n-2)^2$。由于某个局面下的走法数目一般不会超过 70，在利用历史表对走法集合排序时一般的排序算法都能完成任务。

## 4.5 置换表

4.3 节和4.4 节介绍了从不同角度对 α - β 剪枝算法的优化方法，4.3 节通过修改α-β剪枝算法的两个参数使得剪枝算法的收敛速度加快，从而提高剪枝效率。4.4 节通过对走法的排序来优化博弈树的展开顺序，使得算法效率进一步提高。

我们来考察一下递归算法的问题，假设一个计算 n！的函数 f(n)，随着参数 n 的增加计算阶乘的函数运行时间会大幅增加，如果已经计算出 f(20)那么当求 f(30)时若能将之前计算的 f(20)记录下来，计算 f(30)时可以直接查表得到，会节省时间。在中国象棋博弈树搜索中应用这种思想产生的算法叫置换表法。

在博弈树搜索的过程中，存在着已经搜索到的某个局面在随后的搜索中再次出现



中国知网 https://www.cnki.net

的情况，因为在中国象棋中不同的走法序列可能会产生同样的局面，若对已经搜索到的局面记录评估值，下次在遇到相同局面查表便可知道其评估值，这样就会节省时间，减少搜索的节点数这种提高搜索效率的算法就是置换表法。置换表的应用要解决三个问题：

（1）要求存储记录的时间够快，因为存储记录后还要继续搜索博弈树。

（2）查询表的速度要快，能类似于随机存取是最好的。

（3）能在有限的存储空间内进行。

哈希表符合以上要求，一般的数据结构教材都会介绍哈希表技术.为每个可能的局面对应置换表的一项是不可能的，因为中国象棋的局面太多，只能将所有局面映射到一个固定大小的数组中，比如将所有局面映射到 $10^5$大小的表中，在一次搜索中产生的局面数量是有限的，如果定义一个大小合适的置换表，这种做法就是可行的。对每个应该保到置换表中的局面计算出一个置换表的索引值hashindex，索引值一般是 32 位的整数，hashindex%HASHSIZE就是该局面在置换表中的位置。再定义一个 32 位或者 64 位数作为检验项，用来确定是不是要找的项，这个数一般是 64 位的，但是 32 位也可应用，只是部分研究人员认为这样做不够安全，也有用两个 32 位数作为校验项的。不同的=博弈系统置换表中保存的数据不同，不可一概而论，一般的结构如下：

```
struct hashitem
 {
     LONG64 checkitem;//校验项
     int value;//该局面的值
     int depth;//该局面所处的层次
byte hashtype;//节点的类型
 }hashtable[HASHSIZE];//定义哈希表
```

要应用基于哈希原理的置换表还有很多问题需要解决，关键问题如下：

（1）置换表中数据的层次

如前所述置换表的表项中有搜索深度一项，这一项表明了当前的局面在博弈树中的所处的层次，该值越大证明越靠近根节点，反之靠近叶节点，当要搜索的节点在置换表中已经保存了，则比较二者的层次，如果待搜索的节点比置换表中的更靠近根节



中国知网 https://www.cnki.net

点那么应当继续搜索，然后将搜索到的结果保存到置换表中；如果置换表中的节点更靠近根节点就不必再搜索，直接取置换表中的值就可以了。这样做的依据是越靠近根节点的数据更能代表大局利益，看的更全面。

（2）置换表清空

置换表是不是应该在每次搜索结束后都清空有两种做法。第一种是不清空，这种观点认为前次搜索得到的结果对将来的搜索没有不良的影响，不清除就可以使用对后来的搜索是很方便的，而且清除之后表也需要耗费时间。另一种则是清除置换表，在“象眼”程序中的处理方式就是清空置换表。

（3）叶节点的处理

搜索过程中会产生大量的叶节点，对于博弈树搜索过程中的叶节点是不是应该保存到博弈树中要视情况而定，如果局面评估函数较为复杂得到评估函数中包含的知识量很大、计算时间相对长，就应当将叶节点保存的到置换表中，否则不需要保存到置换表中。

Zobrist技术是一种快速生成哈希值的方法，被广泛应用在象棋博弈的程序设计中，不仅可以快速生成索引值还能用它生成校验值。在程序初始化时，建立一个多维数组，在中国象棋中应该是Zob[14][9][10]，用随机数填满这个数组，每当有棋子放到棋盘上就根据棋子的种类和所在的位置得到一个随机值，将棋盘上所有的这样的随机数做异或操作，也可以将走棋方加入其中做异或操作，这样的得到的值hashindex，通过hashindex%HASHSIZE操作就得到了局面在置换表中的存储位置。当移动某个棋子是再次异或它的起点在多维数组中的随机值、异或它的落点在多维数组中的随机值就得到了新的局面的hashindex。Zobrist技术不仅应用在置换表上，长将判负、循环局面检测、开局库和残局库中都有应用。置换表的应用与实验在第五章有更具体的论述。

## 4.6 迭代加深

中国象棋博弈的过程分为开局、中局、残局。在开局和中局时每个棋局状态下都有很多个走法，这时博弈树搜索每层都有多个分支可选。在残局时由于双方经过一段时间的搏杀，可供调动的棋子已经寥寥无几，这时的棋局状态下，走法的数目明显减少。在开局时搜索几层所用的时间，在残局是可能会搜索十几层甚至几十层。在博弈树搜索中单纯的限定搜索的层数是不合理的，这可能导致在开局是要用很久才有结



中国知网 https://www.cnki.net

果，而在残局时却很快就完成搜索，在残局中的搜索层数受到限制严重影响计算机的棋力。

迭代加深先进行 d-1 层的搜索并记录下 d-1 层搜索得到的最佳走法，然后进行 d 层的搜索，在进行 d 层的搜索时优先搜索 d-1 层记录下来的最佳走法，由于 d-1 层和 d 层之间的相似性，d-1 和 d 的最佳走法相同或相似的可能性很大，可以提高剪枝效率，在 d=1 时一般进行全窗口搜索。好的搜索算法一般会在迭代加深中加入置换表、历史表、杀手列表等启发信息。这些在浅层搜索中得到启发信息，将优化深层搜索时博弈树的展开顺序，带来更高的剪枝效率。迭代加深搜索的过程中 d 层搜索的时间开销是 d-1 层搜索的 5～10 倍。所以这点额外的开销并不会过多影响搜索的效率，相反浅层搜索积累的启发信息明显提高了深层搜索的效率。迭代加深的实现和实验测试在第五章将继续讨论。

## 4.7 水平效应与静态搜索

本章前面的知识是从博弈树搜索的角度来论述的，从搜索节点数目和深度深度分析这些算法的效率是可行的。但是有些因素虽然属于博弈树搜索的范畴侧重点却在提高计算机的棋力上，本章本节及本节以下的部分将论述提高棋力并且与搜索相关的技术。

几乎所有的搜索算法都是在指定搜索层数的条件下工作的，这样会导致一些出人意料的错误，可是计算机却还没有察觉到。例如下面几种情况：

（1）搜索算法在搜索 5 层后得到一个车吃炮的走法，计算机会认为这是个最佳走法，事实上这个炮是有车保护的，如果吃了这个跑将会丢掉一个车，因为搜索深度的限制使计算机无法看到这样的局面而犯严重的错。

（2）搜索到 4 层时得到了杀棋，已经无需继续搜索搜索算法返回即可，可是搜索算法的额定层数是 8 层它将继续搜索，这样做不仅仅是浪费时间更有可能搜索到不如意的走法。

在象棋中类似的情况还有很多。这些都是搜索深度固定带来的弊端。这种情况被称为“水平线效应”。那些变化剧烈的局面如果不受搜索层数的限制，继续搜索直到局面缓和为止就可有效克服水平效应，这里所讲的变化剧烈的情况是指有杀棋、将军、对子等存在的局面。静态搜索可以有效改善水平效应，其思想就是当搜索到达指定层



中国知网 https://www.cnki.net

数，对变化不剧烈的叶子节点调用局面评估函数返回评估值，对变化剧烈的节点先不调用局面评估函数，进行有选择性的延伸直到局面平静为止。

静态搜索应用广泛，由于它是搜索算法功能的扩展和补充，很多常规的搜索技术在其中应用并不合适，博弈系统中的静态搜索几乎都不使用置换表、极小窗口、置换表启发等算法，第五张系统实现中将介绍静态搜索在博弈系统中的具体应用和实验。

## 4.8 裁剪算法

### 4.8.1 无害裁剪

在中国象棋中有许多局面在不需要搜的情况下就可以知道它的确切评估值或者范围，存在这样的情况可以把由这个节点构成的树剪掉，这样做不会影响搜索的结果，像这样的裁剪加无害裁剪，常见的有以下几种：

（1）重复裁剪。中国象棋规定如果双方对弈过程中出现重复局面判双方和棋或者一方长打作负，若搜索算法在博弈树中遇到一次重复局面，那么对这个重复局面继续搜索将会再次得到重复的局面，这样的情况就没有必要再搜索下去了。

（2）和棋裁剪。若对弈的双方都没有过河的棋子威胁对方的帅（将），而只剩下士和象的局面必定是和棋的局面，这样的局面也无需再搜索下去了。

### 4.8.2 空着裁剪

在中国象棋对弈中，如果一方连续走棋，一般情况下都会获得很大的利益甚至可以转败为胜。假设计算机所处的某个局面即使让对方连续走棋两步，局势对对方而言依然不利的话，这个分枝就不需要继续搜索了，即可简单的返回 beta 值，这样可以避免由调用局面评估函数进行前向剪枝的危险，并且空着裁剪的实现较为简单。空着裁剪之所以能节省时间在于减少了搜索的深度，搜索深度为 D，深度减少因子为 R，应用空着裁剪后搜索深度就是 D-R。空着裁剪虽然简单，在应用上有很多限制条件，空着裁剪不能连续使用，因为连续使用双方都未走棋，没有意义；在残局时连续走棋不一定能获得利益，因为在残局时可能存在困毙的情况，如图 4.4 所示黑方处于困毙的情形。系统中禁止连续使用空着裁剪，在课题实现的系统中，当局面评估值在 400 以下就不允许做空着裁剪，这样就有效解决了上述问题。将军延伸，或者其它的选择性延伸很好的克服了空着裁剪的不足，在系统中空着裁剪和选择性延伸配合使用既能有效剪枝又能有效提高计算机的棋力。



中国知网 https://www.cnki.net

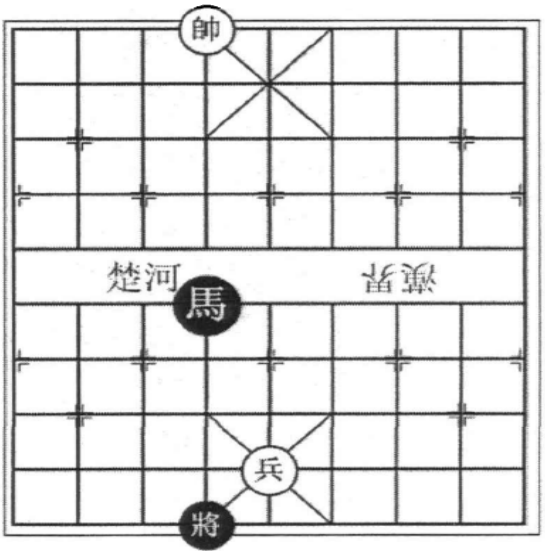



图 4.4 困毙示意图
Fig.4.4 No-Move State



中国知网 https://www.cnki.net

# 第五章 系统实现与实验分析

第二章介绍了棋局的表示、走法的生成，这是一个博弈系统的基础。第三章介绍了局面评估函数，它代表了计算机对当前棋局的理解能力，第四章研究了基于 α-β 剪枝算法的博弈树搜索算法的各种改进策略。本章将介绍实验系统设计，通过实验数据证明应用不同的优化策略对搜索算法的影响。第四章介绍的关于静态搜索的相关理论是博弈树搜索的内容，目的却是在于提高计算机的棋力，本章将综合多种搜索算法优化策略和静态搜索相互配合设计一个拥有一定棋力的综合搜索算法。

## 5.1 走法生成与局面评估函数

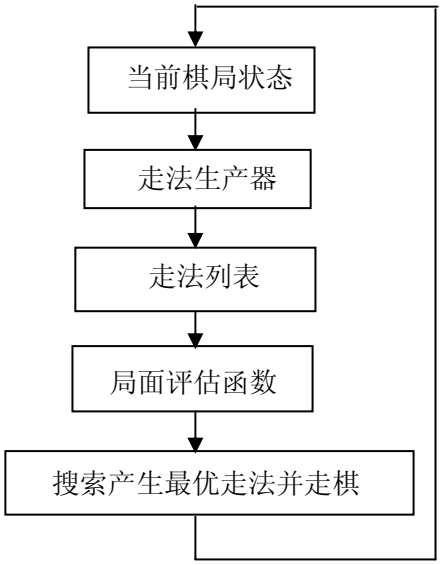



图 5.1 计算机下棋过程
Fig.5.1 Think process of computer

计算机下棋的过程如图 5.1 所示，计算机首先根据当前的棋局状态生成所有的走法，再调用博弈树搜索算法对博弈树进行搜索，当得到博弈树的叶节点时就执行局面评估，根据局面评估值对搜索进行剪枝，最后在根节点得到最优走法并走棋。

系统中应用的棋局状态表示方法和走法表示等相关的数据结构设计在第二章已



中国知网 https://www.cnki.net

经介绍过了，这里不再重复。局面评估函数是计算机棋类博弈的一个研究重点和难点，研究与设计局面评估是函数不是本课题的重点，在系统中仅考虑棋子的位置价值和固定子力价值，在实现上为每个兵种在棋盘的每个落点给出一个位置价值，用一个二维数组 posValue[7][256]表示每个棋子的位置价值。以炮为例，它在棋盘上的位置价值如图 5.2 所示。

```
0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
0,   0,   0, 100, 100,  96,  91,  90,  91,  96, 100, 100,   0,   0,   0,   0,
0,   0,   0,  98,  98,  96,  92,  89,  92,  96,  98,  98,   0,   0,   0,   0,
0,   0,   0,  97,  97,  96,  91,  92,  91,  96,  97,  97,   0,   0,   0,   0,
0,   0,   0,  96,  99,  99,  98, 100,  98,  99,  99,  96,   0,   0,   0,   0,
0,   0,   0,  96,  96,  96,  96, 100,  96,  96,  96,  96,   0,   0,   0,   0,
0,   0,   0,  95,  96,  99,  96, 100,  96,  99,  96,  95,   0,   0,   0,   0,
0,   0,   0,  96,  96,  96,  96,  96,  96,  96,  96,  96,   0,   0,   0,   0,
0,   0,   0,  97,  96, 100,  99, 101,  99, 100,  96,  97,   0,   0,   0,   0,
0,   0,   0,  96,  97,  98,  98,  98,  98,  98,  97,  96,   0,   0,   0,   0,
0,   0,   0,  96,  96,  97,  99,  99,  99,  97,  96,  96,   0,   0,   0,   0,
0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0
```

图 5.2 炮的位置价值

Fig.5.2 Postion value of cannon

PosValue 数组只给出红方的兵种在棋盘上的位置价值，黑方棋子的位置价值就是将该棋子在棋盘上的位置做翻转得到的红方相应兵种的位置价值。定义两个全局变量： int vlRed, vlBlack;每当红方在棋盘的某个位置上增加或去掉一个棋子就执行：vlRed=±posValue[chesskind][postion]，每当黑方在棋盘上曾加或减少一个棋子时就执行：vlBlack=±posValue[chesskind][254-postion]。由于 vlRed 和 vlBlack 分别保存了双方棋子的位置价值的和，所以局面评估函数的很简单，运行速度很快，花费的时间很少，其代码如下：

```
int Evaluate(void) {          // 局面评价函数
    return (sdPlayer == 0 ? vlRed-vlBlack : vlBlack - vlWhite);
  }// sdPlayer== 0 表示红方走棋，1 表示黑方走棋
```

系统中涉及的都是基于负极大值形式的搜索算法，要求局面评估函数对走棋方敏感。



中国知网 https://www.cnki.net

## 5.2 优化策略实现

系统中应用了多种对搜索算法的优化策略，其中包括历史表、置换表、迭代加深等，有些优化策略虽然效果明显但是实现比较简单，在第四章已经做了论述，这里不再赘述，本节介绍较为复杂的优化策略的实现，在后面 5.4 节对应用各种优化策略的算法进行实验，证明它们的效果。

### 5.2.1 历史表实现

历史表中存放的走法的分值，历史表中存放的是兄弟走法中较好的那个或者是产生截断的走法。走法是由一个 16 为的整数表示的，所以定义一个大小为 $2^{16}$的整型数组就能保存所有的走法并且不会产生冲突，系统中用int HistoryTable[65536]保存走法，当需要保存走法mv时其代码为：

```
HistoryTable [mv] = HistoryTable [mv] + ndepth*ndepth;
```

其中ndepth表示剩余的搜索层数，越靠近根节点这个值越大，这里采用的是$ndepth^2$作为走法在历史表中的分值，如果某个走法多次被证明是最佳走法，就将每次得到的分值叠加起来。每当生成一组新的走法时需要根据历史表来排序这些走法，走法的排序调用系统排序函数void qsort( void *base, size_t num, size_t width, int (__cdecl *compare )(const void *elem1, const void *elem2 ) );这里需要一个自定义的排序函数，系统中定义函数CompareHistory()作为qsort()的参数，其代码如下：

```
static int CompareHistory(const void *lpmv1, const void *lpmv2) {
return Search.nHistoryTable[*(int *)lpmv2] - Search.nHistoryTable[*(int *) lpmv1]; }
```

### 5.2.2 置换表实现

置换表保存的是某个局面的分值，所以应用置换表首先要解决局面识别问题。利用 RC4 密码流生成器生成 32 为的随机数来填充 Zobrist 结构体。RC4 密码流生成器代码如下：

```
struct RC4Struct {// RC4密码流生成器
   BYTE s[256];
   int x, y;
   void InitZero(void);     // 用空密钥初始化密码流生成器
   BYTE NextByte(void);    // 生成密码流的下一个字节
```



中国知网 https://www.cnki.net

```
    DWORD NextLong(void);   // 生成密码流的下四个字节
```

RC4结构体生成的随机数可以为今后应用开局库服务，使系统有更好的扩展性，利用RC4结构体给ZobristStruct初始化，其定义如下：

```
struct ZobristStruct {// Zobrist结构
    DWORD dwKey, dwLock0, dwLock1;
    void InitZero(void) {                          // 用零填充Zobrist
        dwKey = dwLock0 = dwLock1 = 0; }
     void InitRC4(RC4Struct &rc4) {                // 用密码流填充Zobrist
        dwKey = rc4.NextLong();
        dwLock0 = rc4.NextLong();
        dwLock1 = rc4.NextLong();}
     void Xor(const ZobristStruct &zobr) { //  执行XOR操作
        dwKey ^= zobr.dwKey;
        dwLock0 ^= zobr.dwLock0;
        dwLock1 ^= zobr.dwLock1; }};
```

dwKey ,dwLock0, dwLock1都是DWORD类型可以保存一个32位数，dwKey用做哈希值，dwLock0和dwLock1用做验证。

```
static struct {// Zobrist表
    ZobristStruct Player;
    ZobristStruct Table[14][256];
} Zobrist;
static void InitZobrist(void) {//  初始化Zobrist表
    int i, j;
    RC4Struct rc4;
    rc4.InitZero();
    Zobrist.Player.InitRC4(rc4);
    for (i = 0; i < 14; i ++) {
        for (j = 0; j < 256; j ++) {
            Zobrist.Table[i][j].InitRC4(rc4);     }}}
```

定义 ZobristStruct zobr 来表示局面，程序开始时 zobr 和 Zobrist 分别调用各自的初始化函数来初始化。初始化完成以后，如果红方在棋盘的某个位置上添加或者去掉



中国知网 https://www.cnki.net

一枚棋子就执行如下操作：zobr.Xor(Zobrist.Table[redchess - 8][position]);如果黑方在棋盘的某个位置上添加或者去掉一枚棋子就执行如下操作：zobr.Xor(Zobrist.Table[blackchess – 9][position]); blackchess，redchess，position 分别表示红方棋子编码，黑方棋子编码，棋子的落点。如果改变走棋方还还要执行 zobr.Xor(Zobrist.Player);操作，这样在置换表项中就不需要保存走棋方的信息了。用两个 32 数验证一个局面也足够安全。Xor()中的操作是异或，这会使运算占用很少的时间。置换表项结构的定义如下：

```
struct HashItem {
    BYTE ucDepth, ucFlag;//深度，值的类型
    short svl;//局面分值
    WORD wmv;//走法
    DWORD dwLock0, dwLock1; };//局面验证信息
```

定义HashItem HashTable[HASH_SIZE]; 表示置换表，其中宏HASH_SIZE表示置换表的大小，其值为 $2^{25}$。置换表的查询与保存函数的流程如图 5.3 和 5.4 所示。

### 5.2.3 迭代加深实现

迭代加深的实现关键在时间控制和迭代过程中的有用信息的保存，其实现代码如下：

```
memset(HistoryTable, 0, 65536 * sizeof(int));       // 清空历史表
memset(mvKillers, 0, LIMIT_DEPTH * 2 * sizeof(int)); // 清空杀手走法表
memset(HashTable, 0, HASH_SIZE * sizeof(HashItem));   // 清空置换表
t = clock();              // 初始化定时器
nDistance = 0; // 初始步数
// 迭代加深过程
for (i = 1; i <= LIMIT_DEPTH; i ++) {
        vl = Search (-MATE_VALUE, MATE_VALUE, i);
    // 搜索到杀棋，就终止搜索
    if (vl > WIN_VALUE || vl < -WIN_VALUE) {
        break;
    }
```



中国知网 https://www.cnki.net

```
// 超过一秒，就终止搜索
if (clock() - t > CLOCKS_PER_SEC) {
    break;  }}
```

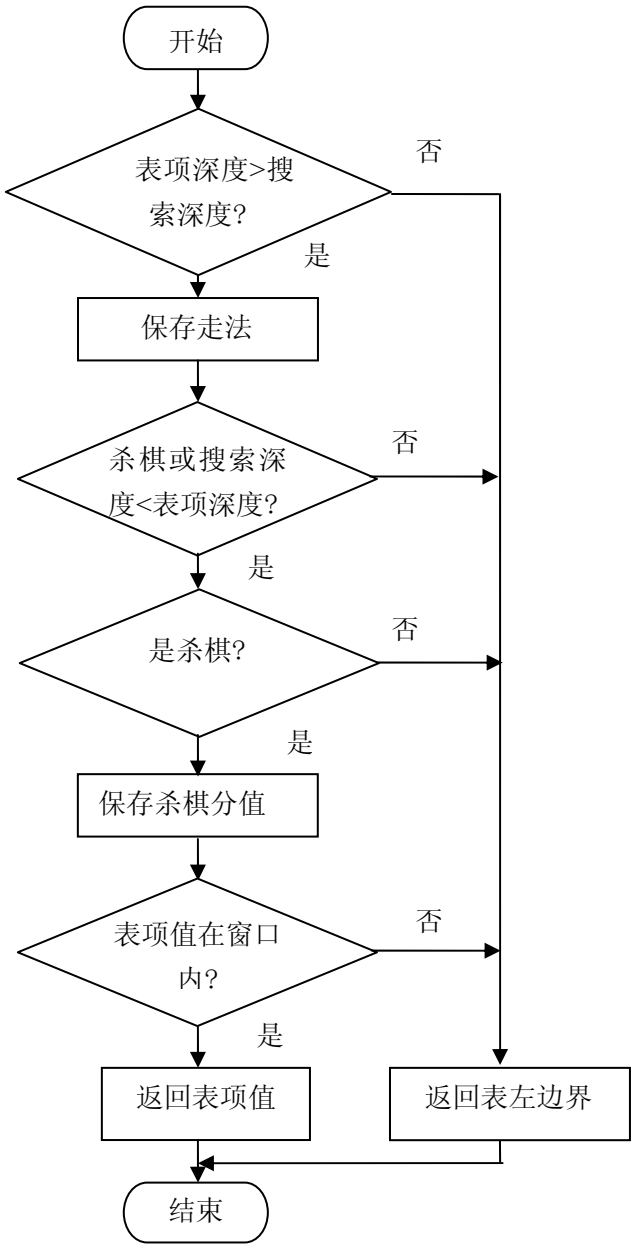



图 5.3 查询置换表流程图

Fig.5.3 Flowchart of query transposition table



中国知网 https://www.cnki.net

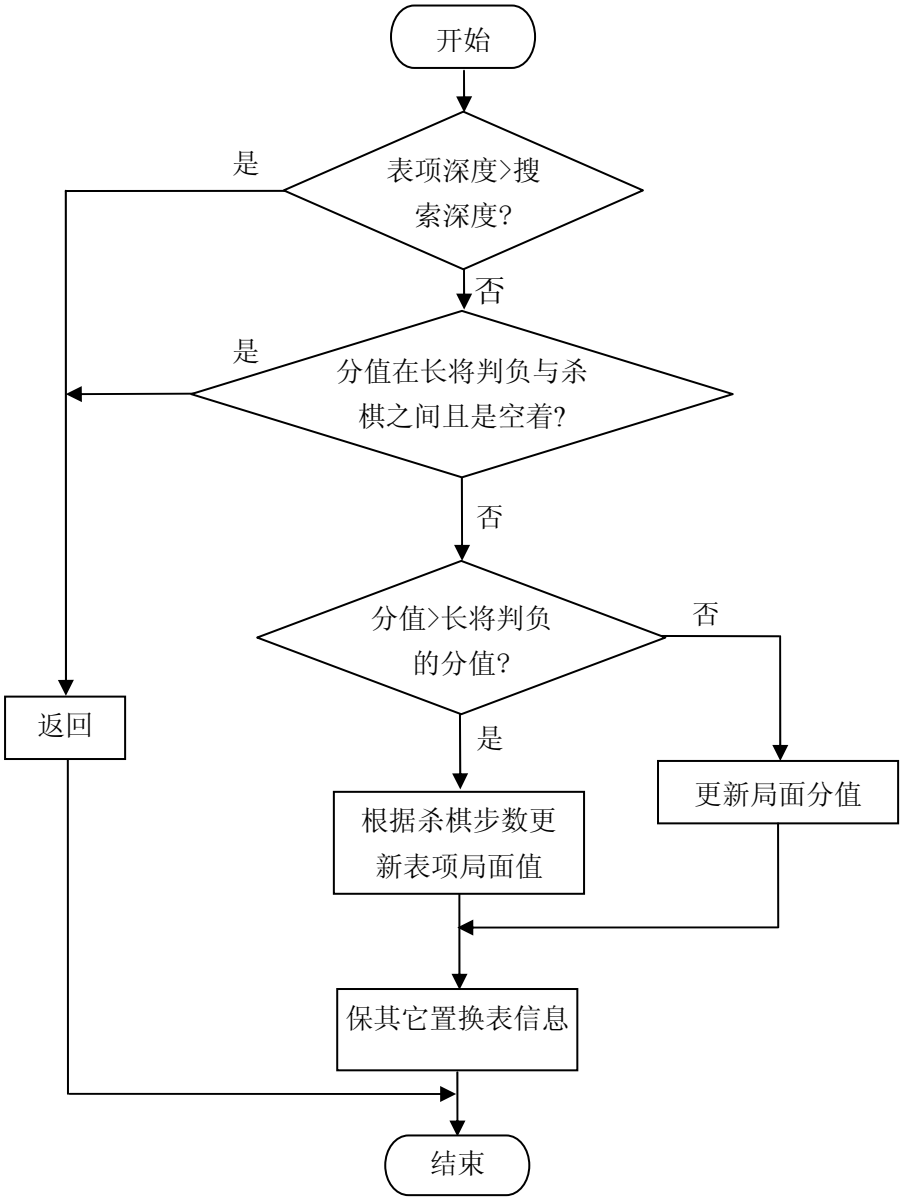



图 5.4 保存置换表流程图
Fig.5.4 Flowchart of store transposition table

在迭代开始之前要设置一个时间用库函数clock()作为定时器计时开始，还要将历史表，杀手列表，置换表等上次搜索保存的信息清空，因为这些信息对下次搜索可能会产生不良的影响。每次迭代完成都检查是否超过设定的时间如果超过接跳出迭代循环，返回搜索的结果。



中国知网 https://www.cnki.net

### 5.2.4 静态搜索实现

在静态搜索中实现对将军的局面的延伸搜索，在不将军的局面下实现吃子走法的延伸，对吃子走法的排序采用 MVV/LVA 排序。

```
// MVV/LVA每种子力的价值
static BYTE MvvLva[24] = {
  0, 0, 0, 0, 0, 0, 0, 0,
  5, 1, 1, 3, 4, 3, 2, 0,
  5, 1, 1, 3, 4, 3, 2, 0
};
```

MvvLva表示每种棋子的MVV/LVA价值，0是填充位，5，1，1，3，4，3，2分别代表帅、仕、相、马、车、炮、兵的MVV/LVA价值，由于对帅的攻击是对弈的目的，所以帅的MVV/LVA价值最大。定义成三排数据是为了使用方便。

```
// 求MVV/LVA值
inline int MvvLva(int mv) {
  return (MvvLva[Squares[DST(mv)]] << 3) - cucMvvLva[Squares[SRC(mv)]];
}
```

对于一个吃子的走法，其MVV/LVA价值是走法落点棋子的MVV/LVA价值的8倍减去走法起点棋子的MVV/LVA价值，这样可以综合考虑攻守双方的价值。

```
// "qsort"按MVV/LVA值排序的比较函数
static int CompareMvvLva(const void *lpmv1, const void *lpmv2) {
  return MvvLva(*(int *) lpmv2) - MvvLva(*(int *) lpmv1);
}
```

CompareMvvLva()的作用是对吃子走法进行排序，以便优先展开最佳的节点，提高剪枝的效率。

被将军的局面可行的走法较少，可以在静态搜索中将被将军局面下的所有走法都生成，根据历史表对走法进行排序，这样可以有效加强计算机对杀棋的判断。

静态搜索实现的代码如下：

```
static int SearchQ(int vlAlpha, int vlBeta) {
  int i, nGenMoves;
  int vl, vlBest;
```



中国知网 https://www.cnki.net

```
int mvs[MAX_GEN_MOVES];
//到达极限深度就返回局面评价
if (nDistance == LIMIT_DEPTH) {
   return Evaluate();}
   //初始化最佳值
vlBest = -MATE_VALUE; //  这样可以知道，是否一个走法都没走过(杀棋)
if (InCheck()) {
   //   如果被将军，则生成全部走法
   nGenMoves =GenerateMoves(mvs);
   qsort(mvs, nGenMoves, sizeof(int), CompareHistory);
} else {
   //   如果不被将军，先做局面评价
   vl =Evaluate();
   if (vl > vlBest) {
      vlBest = vl;
      if (vl >= vlBeta) {
         return vl;    }
            if (vl > vlAlpha) {
         vlAlpha = vl;    }    }
            //   如果局面评价没有截断，再生成吃子走法
   nGenMoves = GenerateMoves(mvs, GEN_CAPTURE);
   qsort(mvs, nGenMoves, sizeof(int), CompareMvvLva);
}
//   逐一走这些走法，并进行递归
for (i = 0; i < nGenMoves; i ++) {
   if (MakeMove(mvs[i])) {
      vl = -SearchQ (-vlBeta, -vlAlpha);
      UndoMakeMove();
      // 8. 进行Alpha-Beta大小判断和截断
      if (vl > vlBest) {       //  找到最佳值
      vlBest = vl; // "vlBest"就是目前要返回的最佳值，可能超出Alpha-Beta边界
```



中国知网 https://www.cnki.net

```
if (vl >= vlBeta) { // 找到一个Beta走法
  return vl; }              // Beta截断
        if (vl > vlAlpha) {
  vlAlpha = vl; } } } }     // 缩小Alpha-Beta边界
//所有走法都搜索完了，返回最佳值
return vlBest == -MATE_VALUE ? nDistance - MATE_VALUE : vlBest; }
```

加入静态搜索可以有效克服水平效应，虽然在搜索算法中加入静态搜索会增加搜索的节点数但是这样做会有效提高计算机的棋力。

## 5.3 系统实现

### 5.3.1 系统功能介绍

本课题的博弈系统是在 visual studio 2008 下实现的，采用 c++语言编程实现。利用 vc++提供的对话框作为主窗口，实现了以下几部分功能：

（1）博弈树搜索算法的优化策略很多，应用不同的优化策略对搜索算法影响也不同，为了验证优化策略的效率，设计了五个搜索算法：Fail-Soft+历史表；Fail-Soft+历史表+空着裁剪；Fail-Soft+历史表+空着裁剪+置换表；Fail-Soft+历史表+空着裁剪+置换表+迭代加深；PVS+历史表+空着裁剪+置换表+迭代加深。实现了对这些搜索引擎的搜索层数的设置，进行人际对弈，观察性能。设置搜索算法的对话框如图 5.5 所示。可以在搜索算法设置对话框中选择搜索算法和它的搜索深度。人机对弈的界面如图 5.6 所示。人机对弈的流程如图 5.7 所示。

（2）Fail-Soft+历史表+空着裁剪+迭代加深；Fail-Soft+历史表+空着裁剪+置换表+迭代加深；PVS+历史表+空着裁剪+置换表+迭代加深三个搜索算法加入静态搜索提高棋力，实现搜索算法之间的对弈，观察结果。计算机之间对弈的流程如图 5.7 所示。

（3）实现双方的走法棋谱的生成和显示；实现了显示搜索引擎走每步棋的叶节点数目显示，实现统计并显示计算机走每步棋所用的时间的显示，实现统计并显示搜索过程中置换表匹配成功次数。计算机之间对弈界面如图 5.8 所示。



中国知网 https://www.cnki.net




图 5.5 搜索算法设置
Fig.5.5 Search algorithm set

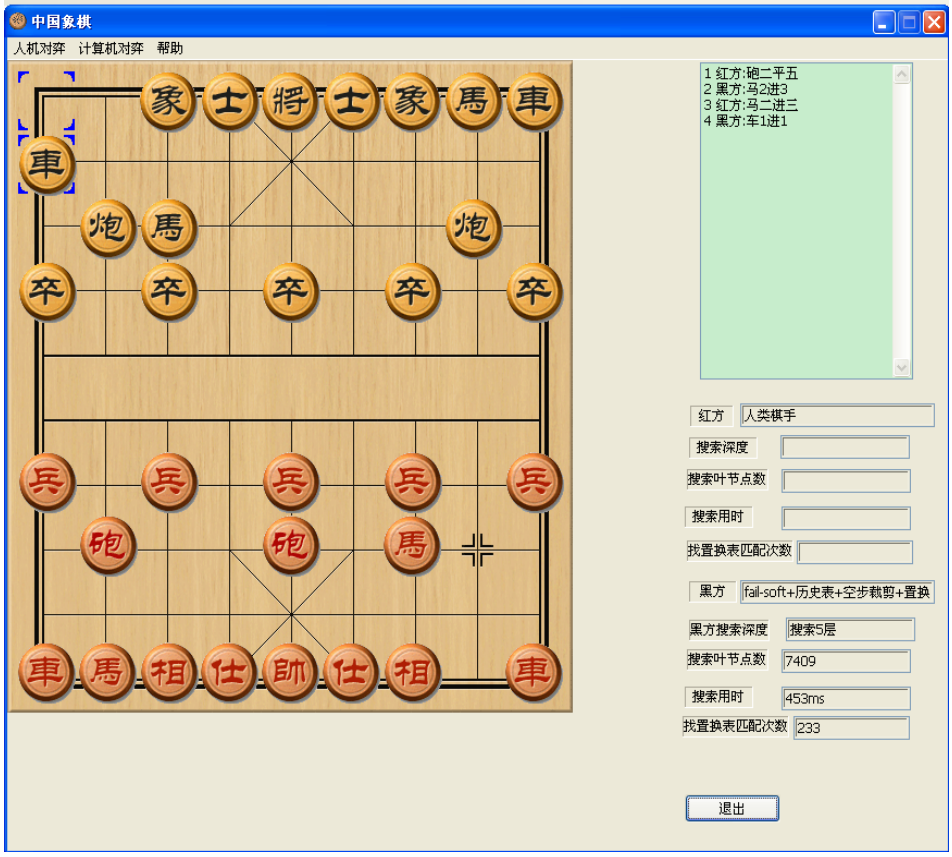



图 5.6 人机对弈
Fig.5.6 People VS computer



中国知网 https://www.cnki.net

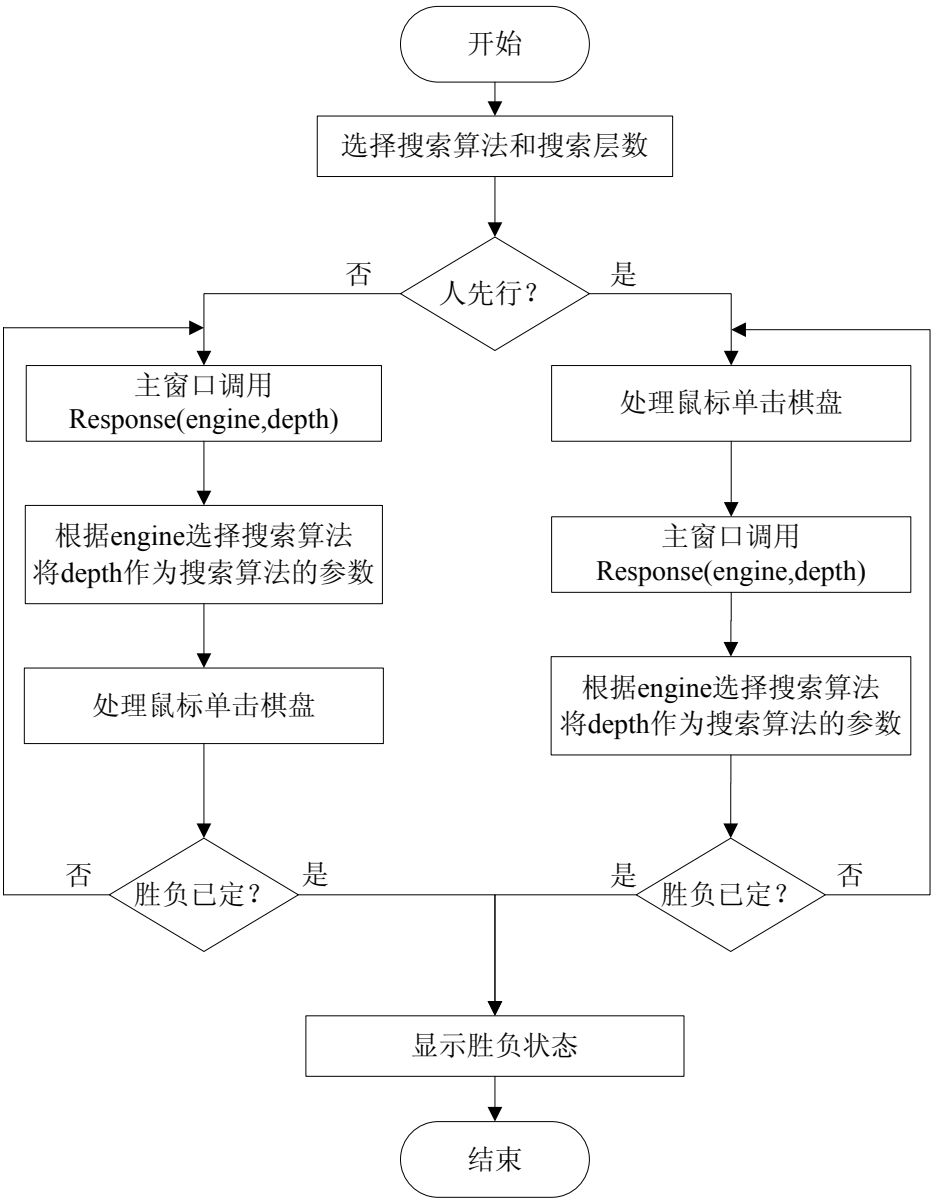



图 5.7 人机对弈的流程图
Fig.5.7 Flowchart of people VS computer

### 5.3.2 综合搜索算法

5.4 节将通过实验说明综合搜索算法 PVSHNTI 的效率最高，是以 PVS 为基础，加入历史表、空着裁剪、置换表法、迭代加深等优化策略的综合搜索算法。为 PVSHNTI 加入静态搜索得到搜索算法 PVSHNTIQ。从总体上看是由三个函数构成的，这个三个函数是：SearchQ()；SearchPVSHNTIQ()；PVSHNTIQ()。SearchQ()完成静态搜索，在 5.2.4 节已经介绍，PVSHNTIQ()中实现迭代加深的时间控制，5.2.3 节介绍的代码段中将 Search()替换为 SearchPVSHNTIQ()。SearchPVSHNTIQ()的代码如下：



中国知网 https://www.cnki.net

```
static int SearchPVSHNTIQ(vlAlpha, vlBeta, int nDepth, BOOL bNoNull = FALSE)
  {
   if (nDepth <= 0) {
        return SearchQ(vlAlpha, vlBeta);  }//静态搜索
  if (nDepth== LIMIT_DEPTH)
       return pos.Evaluate();//达到极限深度返回局面评估值
   vl = ProbeHash(vlAlpha, vlBeta, nDepth, mvHash); //查询置换表走法
   if (vl > -MATE_VALUE) {
        return   vl;  }
   if (!bNoNull && !pos.InCheck() && pos.NullOkay()) {//   尝试空着裁剪
        pos.NullMove();
   vl =-SearchPVSHNTIQ(-vlBeta,1-vlBeta,nDepth-NULL_DEPTH-1, NO_NULL);
        pos.UndoNullMove();
        if (vl >= vlBeta) {
            return vl; }}
   nHashFlag = HASH_ALPHA;//   初始化最佳值和最佳走法
   vlBest = -MATE_VALUE; //  这样可以知道，是否一个走法都没走过
   mvBest = 0;
   Sort.Init(mvHash);    //   初始化走法排序结构
        while ((mv = Sort.Next()) != 0) {//   逐一走这些走法进行递归
        if (pos.MakeMove(mv)) {
            nNewDepth = pos.InCheck() ? nDepth : nDepth - 1;
            if (vlBest == -MATE_VALUE) {//PVS 窗口剪枝
                vl = - SearchPVSHNTIQ (-vlBeta, -vlAlpha, nDepth-1);
            } else {
                vl = - SearchPVSHNTIQ (-vlAlpha - 1, -vlAlpha, nDepth-1);
                if (vl > vlAlpha && vl < vlBeta) {
                    vl = - SearchPVSHNTIQ (-vlBeta, -vlAlpha, nDepth-1); }     }
            pos.UndoMakeMove();
            //进行 Alpha-Beta 大小判断和截断
            if (vl > vlBest) { //  找到最佳值
```



中国知网 https://www.cnki.net

```
vlBest = vl; //是目前要返回的最佳值，可能超出 Alpha-Beta 边界
                if (vl >= vlBeta) { //  找到 Beta 走法
                    nHashFlag = HASH_BETA;
                    mvBest = mv;       // Beta 走法要保存到历史表
                    break;     }            // Beta 截断
                if (vl > vlAlpha) { //  找到一个 PV 走法
                    nHashFlag = HASH_PV;
                    mvBest = mv;    // PV 走法要保存到历史表
                    vlAlpha = vl;}}} }    //  缩小 Alpha-Beta 边界
if (vlBest == -MATE_VALUE) {//把最佳走法保存到历史表，返回最佳值
    return    pos.nDistance - MATE_VALUE;}//是杀棋根据杀棋步数给出评价
RecordHash(nHashFlag, vlBest, nDepth, mvBest);  //  记录到置换表
if (mvBest != 0) {
    //  如果不是 Alpha 走法，就将最佳走法保存到历史表
    SetBestMove(mvBest, nDepth);
    if (pos.nDistance == 0) {
    Search.mvResult = mvBest;    }}//  搜索根节点时，总是有一个最佳走法
return vlBest;}
```



中国知网 https://www.cnki.net

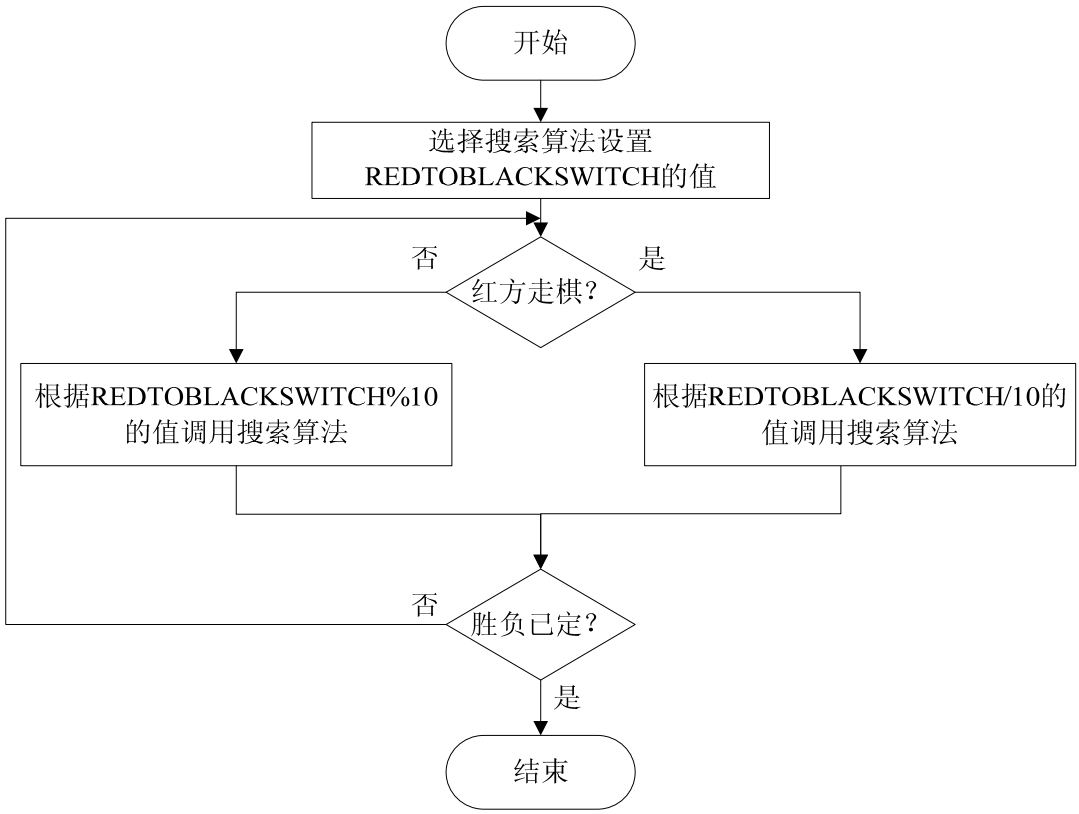



图 5.7 计算机之间对弈的流程图
Fig.5.7 Flowchart of computer VS computer

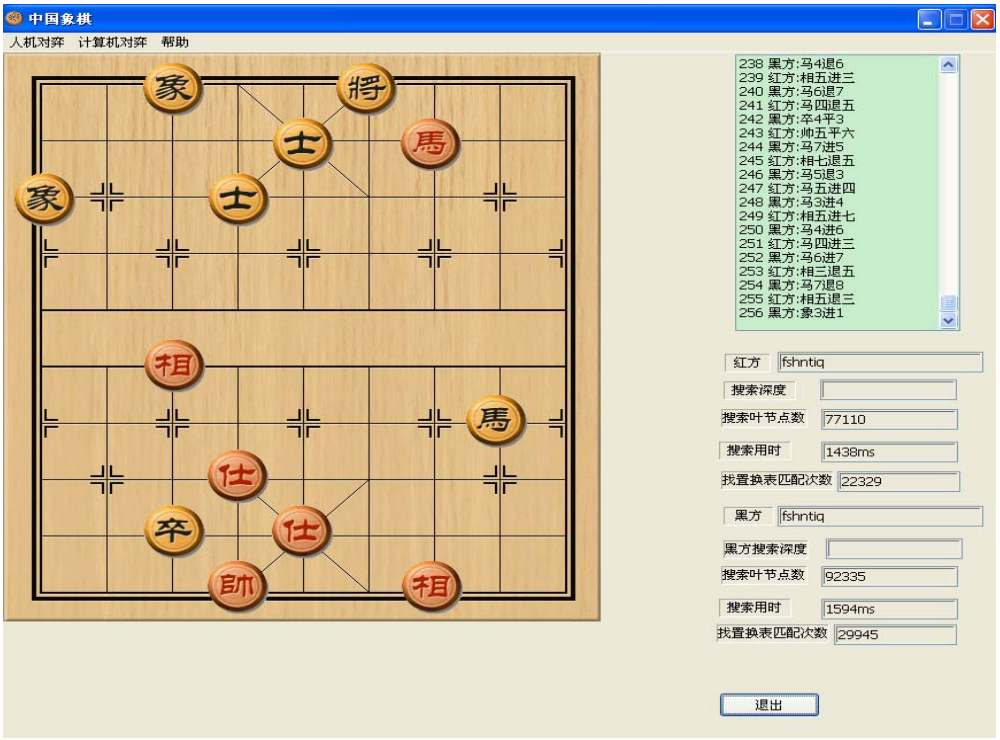



图 5.8 计算机之间对弈
Fig.5.8 Computer VS computer



中国知网 https://www.cnki.net

## 5.4 实验分析

基于 α-β 剪枝的博弈树搜索算法有多种改进策略，本节将对应用不同优化策略的搜索算法进行性能测试，从而证明了拥有多项优化策略的综合搜索算法无论从剪枝效率还是棋力上都优于其它的搜索算法。用于实验的机器的配置为 AMD Athlon(tm) 64 ×2 Dual Core Processor 4000+ 2.11GHz，896MB 内存。结合不同的改进策略得到五个搜索算法，为表述方便，用字母代替优化策略，搜索算法的名字代表了加入的优化策略。

表 5.1 字母与优化策略的对应关系

Table5.1 Correspondence between Letter and optimization strategy

| 字母 | FS | H | N | T | I | PVS | Q |
|---|---|---|---|---|---|---|---|
| 优化策略 | Fail-Soft 算法 | 历史表法 | 空着裁剪 | 置换表法 | 迭代加深 | 极小窗口搜索算法 | 静态搜索 |

下文中的 FSH 表示以 Fail-Soft 算法为基础加入了历史表法的综合搜索算法。其它搜索算法和 FSH 的命名方法一样。

表 5.2 列出了应用不同优化策略得到的五个搜索算法的搜索层数和节点数的关系。测试的数据是开局 5 步搜索的叶节点的平均数，表中“—”表示由于搜索的时间数目或节点数目太大没有记录。

表 5.2 开局 10 步算法搜索叶节点数与搜索层数的关系

Table5.2 The relationship between the amount of search layer nodes and the amout of search layer in the start 10 steps

| 层数 / 算法 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| FSH | 197 | 2371 | 10185 | 155834 | 725242 | 7873289 | — | — | — |
| FSHN | 288 | 1056 | 8268 | 18516 | 184624 | 576346 | 4529120 | — | — |
| FSHNT | 236 | 1018 | 3934 | 10533 | 64376 | 16653 | 780233 | 2036164 | 24042391 |
| FSHNTI | 241 | 832 | 3547 | 11346 | 49012 | 153742 | 532290 | 1429967 | 5652073 |
| PVSHNTI | 331 | 1442 | 4365 | 12029 | 58537 | 120294 | 475786 | 1310443 | 3514383 |



中国知网 https://www.cnki.net

观察表可以得出随着搜索深得的增加，搜索的叶节点数成倍增加，由于剪枝算法的加入使得节点不是以 $40^n$增长。FSH搜索算法加入了历史表，优化了博弈树的节点排序使得算法效率的效率的到明显提高。FSHN搜索算法在FSH基础上加入空着搜索，使得搜索算法的效率提高了几十倍。FSHNI算法中加入了置换表，比较第三行和第四行的数据可知，在浅层搜索中FSHNI和FSHN的效率相似，但是搜索层次在四层以上，置换表明显提高了搜索算法的效率，这是因为随着搜索的加深置换表中保存的好的走法数量将增加，使得命中置换表的次数增加，减少了搜索的节点数从而提高了搜索算法的效率。观察表 5.1 的最后两行发现PVSHNTI搜索算法在搜索层数在 5 层以下（包括 5 层）节点数略多于FSHNTI搜索算法，搜索层数在 5 层以上效率高于FSHNTI搜索算法。通过实验发现基于PVS的PVSHNII搜索算法运行非常不稳定，原因在于PVS搜索设定的窗口很小，这样使得算法剪枝效率提高，也增加了算法执行失败的可能性，但是从总体上看PVSHNTI的效率高于FSHNTI。综上所述从搜索的叶节点数的角度看，PVSHNTI的搜索效率最高。

表 5.3 列出了五个搜索算法搜索不同层的执行时间。其中时间的单位是毫秒，“—”表示时间太长没有统计的意义。

表 5.3 开局 10 步算法搜索的时间与层数的关系

Table5.3 The relationship between search time and search layer in the start 10 steps

| 时间 / 算法 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| FSH | 4 | 17 | 111 | 1182 | 8354 | 70259 | — | — | — |
| FSHN | 9 | 11 | 67 | 165 | 1454 | 4512 | 47549 | — | — |
| FSNT | 521 | 487 | 508 | 568 | 942 | 1631 | 5918 | 13963 | 135391 |
| FSNTI | 484 | 490 | 583 | 576 | 736 | 1423 | 3923 | 11213 | 41953 |
| PVSHTI | 525 | 572 | 592 | 623 | 889 | 1272 | 3450 | 10413 | 23643 |

观察表可得出随着搜索的层数的增加，搜索时间也增加，FSH 搜索 6 层时所用时间已经达到了 8354 毫秒。随着置换表的应用使得搜索算法在浅层搜索中的时间增加，分析导致这种结果的原因是由于在浅层搜索中博弈树的规模小存入置换表的局面在随后的搜索中并没有出现或者出现的并不多，从而使得置换表中存入的局面在随后的



中国知网 https://www.cnki.net

搜索中部没有起到作用，白白浪费了宝贵的时间，随着博弈树搜索层数的增加，出现相同局面的可能性也增加，因而置换表在搜索深度大于 5 层时起到的作用更为显著。迭代加深同样使得浅层搜索的时间增加。

表 5.4 给出了 FSHNT 搜索算法搜索深度为 8 层，开局 10 步置换表大小与匹配次数的关系，从表中可以看出随着置换表的增大置换表匹配次数也在增加，分析原因是置换表增大使冲突降低，从而增加了匹配的次数。置换表大小为 64K 和大小为 1M 时的匹配次数相差不大，说明对于 FSHNT 搜索算法 64K 的置换表已经足够大，再增加置换表的大小对提高置换表的匹配次数是没有意义的。表 5.3 给出的搜索算法搜索 10 层时最快的也要 23 秒，所以系统中的置换表至少要 1M。系统是在 visual studio 2008 开发环境用 c++语言设计的，在不断增大置换表的过程中发现置换表大小为 32M 时系统可以正常运行，且能保证系统需求，当置换表的大小为 64M 时系统运行很慢几乎处于死机状态，当置换表大小为 128M 时系统报错，因为系统支持的数组的最大容量为 2G 字节，置换表的表项是结构体类型所以当置换表大小为 128M 时会报错。鉴于现在 PC 机的内存一般在 2G 左右，为保证置换表的效率系统中使用 32M 的置换表。这样既保证效率也是系统可以正常运行。表 5.2 和表 5.3 都是在 32M 置换表下进行的实验。根据机器的配置和搜索的深度来设计置换表的大小是正确的做法。

表 5.4 FSHNT 搜索 8 层置换表大小与匹配成功次数
Table5.4 Relationship between the amout of translationTable and match time

| 置换表大小 | 256 | 1K | 4K | 16K | 64K | 1M |
|---|---|---|---|---|---|---|
| 匹配成功次数 | 1180 | 3852 | 16099 | 30724 | 43784 | 43582 |

为提高计算机的棋力需要对博弈树某些重要的分支做延伸，实验测试 PVSHNTI 与 PVSHNTIQ 的对弈，二者各先走 5 次，10 次对弈的结果都是 PVSHNTIQ 获得胜利。这表明应用静态搜索后搜索算法的棋力得到了提高，由于在静态搜索中只对将军和吃子的走法进行搜索，增加的搜索量不会严重影响搜索用时，实验中没有明显的时间延长。实验 PVSHNTIQ 与 FSHNTIQ 的对弈，PVSHNTIQ 胜四局，平三局，输三局，证明 PVSHNTIQ 的棋力高于 FSHNTIQ，实验过程中发现搜索算法相同的情况下，红方的胜率高于黑方，这是由于在棋类博弈中先行者更占优势。



中国知网 https://www.cnki.net

# 第六章 结论

本文对博弈系统的各个组成部分做了研究，包括数据结构的设计，走法生成的优化；局面评估函数相关理论的研究与实现；重点研究了基于 α-β 剪枝算法的博弈树搜索算法及其优化策略，完成了人机对弈和计算机之间对弈的系统实现，通过实验证明了综合多种优化策略对搜索算法改进的有效性。通过对课题的研究，得出以下结论：

（1）在棋局状态表示上，采用大小为 256 的一维数组，这种表示方法使程序中一些基本函数的代码量降低。通过预置表和模板匹配的方法，加快了走法生成的速度。

（2）在到达极限搜索深度时调用局面评估函数，考虑棋子固定子力价值和棋子位置价值，既保证对局面评估准确性又能快速返回评估值。

（3）在搜索算法的研究上，对 Fail-Soft alpha-beta 算法和 PVS 算法进行优化，采用空着裁剪、置换表、历史表、迭代加深。实验表明优化策略的加入显著提高了搜索算法的剪枝效率。为克服搜索算法的水平效应，采用静态搜索对重点路线做延伸搜索，明显提高了搜索算法的棋力。

虽然本文实现了一个具备一定棋力的博弈系统，但是由于本人水平、精力有限以及象棋博弈本身的复杂性，还存在不足之处。

（1）系统中应用的评估函数的知识量较少，应该为局面评估函数赋予更多的知识甚至使其具有自学习的能力。

（2）在实验过程中发现在开局时计算机经过长时间的运算得到的却是一个蹩脚的走法，所以开局库的应用不仅能提高计算机的棋力还能节省计算机开局时的搜索时间。



中国知网 https://www.cnki.net

# 参考文献


[1] Nils. J Nilsson. 人工智能. 北京：机械工业出版社，2000. 63～65.
[2] 蔡自兴，徐光佑. 人工智能及其应用. 北京：清华大学出版社，2003. 12～14.
[3] 卫萌菡，方旭明. 基于博弈论的无线宽带网络协作资源管理，计算机应用，2010，30(05)：79～83.
[4] 舒晓苓. 游戏怪物 NPC 的人工智能分析，电脑知识与技术，2008(09)：15～16.
[5] 电脑国际象棋简史. http://www.xqbase.com/computer/history.htm. 2008-6-5.
[6] 易学君. “深蓝”的力量-卡斯帕罗夫与“深蓝”的人机大战. http://www.douban.com /note /94332783/. 2005-8-14.
[7] Frederic Friedel. 国际象棋人机博弈史. http://.chessit.netUfile_topic/ computerehess/c_briefhistory.html . 2007-8-19
[8] 象棋百科全书. 中国象棋引擎象眼网站. http://www.elephantbase.net. 2007-10-25.
[9] 北方网. http://news.enorth.com.cn/system/2010/06/12/004766036.shtml. 2009-8-17.
[10] 棋天大圣网站. http://www.neuchess.com/. 2006-9-9.
[11] 计算机博弈大赛背景. http://www.robochina.cn/newsInfo.aspx?id=142. 2010-12-9.
[12] 北京理工大学. http://ss.bit.edu.cn/AppWeb/News/news.aspx?id=43. 201-11-28.
[13] 许景婷. 企业税收筹划行为的博弈研究. 工业技术经济，2006，25(11)：101～102.
[14] 田开友，郭光勇，范小云. 武汉理工大学学报（社会科学版），2003，24(06)：79～81.
[15] 王一非. 具有自学习功能的计算机象棋博弈系统的研究与实现：（硕士学位论文）. 哈尔滨：哈尔滨工程大学，2007.
[16] 危春波. 中国象棋博弈系统的研究与实现：（硕士学位论文）. 昆明：昆明理工大学，2008.
[17] “棋天大圣”课题组. 中国象棋计算机博弈问题研究系列论文. 沈阳：东北大学人工智能与机器人研究所，2006.
[18] 谢国. 中国象棋机器博弈数据结构设计与搜索算法研究：（硕士学位论文）. 西安：西安理工大学，2008.
[19] 毕津滔. 中国象棋对弈机器人控制系统研究：（硕士学位论文）. 哈尔滨：哈尔滨理工大学，2009.
[20] 周玮，王水涛，孙旸. 中国象棋计算机博弈中的一种数据结构方法，计算机工程与应用，2006，45(35)：26~27.
[21] 翟乃强. 改进的中国象棋棋盘识别方法，计算机应用，2010，29(04)：35～36.
[22] 周玮，张啧，周静怡. 基于对弈局势的二次估值方法，系统仿真学报，2006，27(09)：41～43.
[23] 王小春. PC 游戏编程（人机博弈）. 第一版.重庆：重庆大学出版社，2002.
[24] 鲍鹏，高珩，王伟. 中国象棋的编程设计，电脑学习，2009(05)：67～68.
[25] 谢艳茹. 中国象棋计算机博弈数据结构与评估函数的研究和实现：(硕士学位论文). 西安：西安理工大学，2008.
[26] 樊莉，孙继银，王勇. 人工智能中的A*算法应用及编程，微机发展，2003，23(05)：56～58.
[27] 王骥. 博弈树搜索算法的研究及改进：（硕士学位论文）. 浙江：浙江大学，2006.
[28] 张振，顾治华. 机器博弈及其搜索算法的研究，电脑知识与技术，2008，18(24)：74～76.
[29] 李红，吴粉侠，刘小豫. 博弈树搜索算法研究，长春工程学院学报（自然科学版），2007，11(02)：23～25.
[30] D.E. Knuth, R. W. Moore. An Analysis of Alpha-Beta Pruning. Artificial Intelligence, 1975, 06(04):293~326.




中国知网 https://www.cnki.net

[31] 岳金朋，冯速. 中国象棋 Alpha-Beta 搜索算法的研究与改进，北京师范大学学报（自然科学版），2009，54(02)：31～34.
[32] 王骄，王涛，罗艳红等. 中国象棋计算机博弈系统评估函数的自适应遗传算法实现，东北大学学报（自然科学版），2005，26(05)：949～952.
[33] 褚泽永，黄鸿，黄远灿. 浅析中国象棋计算机博弈关键技术，中国机器博弈学术研讨会，北京，2006：48～50.
[34] 舒康元，胡福乔. 中国象棋计算机博弈引擎改进，微计算机信息，2009，27(06)：56～58.
[35] 岳金朋，冯速. 博弈树搜索算法概述，计算机系统应用，2009，36(09)：47～50.
[36] 危春波，王海瑞，文乔农. 博弈树搜索算法的分析与实现，科技广场，2007，25(05)：30～31.
[37] 张聪品，刘春红，徐久成. 博弈树启发式搜索的 α-β 剪枝技术研究，计算机工程与应用，2008，35(16)：56～57.
[38] A Storage Structure and Capture Judging Algorithm of Realizing the Computer Game Program of Surakarta Chess. Proceedings of the 2011 Chinese Control and Decision Conference (CCDC), 2011,114~115.
[39] 王赠凯，吕维先. 机器博弈搜索技术分析，软件导刊，2007，6(03)：25～26.
[40] 焦尚彬，刘丁. 博弈树置换表启发式算法研究，计算机工程与应用，2010，37(06)：48～49.
[41] 王骐，孙建伶. 基于优化迭代的博弈树算法，计算机应用与软件，2008，25(02)：61～62.
[42] 张振，庞海. 机器博弈及其搜索算法的研究，软件导刊，2008，7(07)：10～11.
[43] Discussion on Chinese Chess Decision-making System with Emotion. Proceedings of the 2011 Chinese Control and Decision Conference (CCDC), 2011,87~88.
[44] 魏钦刚，王骄，徐心和等. 中国象棋计算机博弈开局库研究与设计，智能系统学报，2007，2(01)：61～63.
[45] Dynamic Opening-book in Computer Games. Proceedings of the 2011 Chinese Control and Decision Conference(CCDC), 2011: 145~147.
[46] Computer Games Are an Efficient Tool for Event Game Theory. Proceedings of the 2011 Chinese Control and Decision Conference(CCDC), 2011: 92~93.
[47] Plaat A, Schaeffer. J Pijls. W.Best-First-Fixed-Depth Minmax Algorithms. Artificial Intelligence, 1998,7(02): 255~293.
[48] Qian L The Evolution of Mulan: Some Studies In Game-tree pruning And Evaluation Function In The Game Amazons.Master of Science Thesis, 2003, 9(12): 79~82.
[49] Rivest R .Game Tree Searching by MinMax Approximation. Artificial Intelligence,1998,34(01): 41~44.
[50] Charles E, Leiserson D, Aske P. Programing Parallel Applications in Cilk.SLAM News,1998,31(04): 6~7.
[51] Ken Auer, Erik M, Gareth R.The Rules of the Game. XP/Agile Universe, 2003, 10(02): 35~42.



中国知网 https://www.cnki.net

# 在学研究成果

［1］田翠华，宋兴亮，朱顺痣，孔丽新．中国象棋计算机博弈算法研究与实现，信息技术，2011(12)．（涉及论文第五章内容，已发表）



中国知网 https://www.cnki.net

# 致　　谢

首先，我要感谢我的导师田翠华，感谢田老师两年多来对我的学习和生活中无微不至的关心，感谢她对我硕士论文研究工作所给予的细心指导。感谢她以渊博的学识，豁达的态度和智慧的思想指引我走过人生中这一重要阶段。

其次，真诚地感谢实验室的同学们和室友，感谢王龙吉、刘胜旺、董事超、姜本臣等同学，感谢他们给予过我的帮助和关怀。

最后要感谢的是我的家人，他们给予了我最无私的关怀，我们共享着生活中的快乐和对生活的理解。我永远爱他们。



中国知网　https://www.cnki.net